

LLM Agents Are Latent Context Managers: Eliciting Self-Managed Context via State Proprioception

Binyan Xu^{1,2,*}, Haitao Li^{2,†}, Kehuan Zhang¹

¹The Chinese University of Hong Kong, ²LIGHTSPEED

{binyxu, khzhang}@ie.cuhk.edu.hk, 729156675@qq.com

*Work done during an internship at Tencent. †Corresponding author.

Project Page <https://vista-agent.github.io/>
Code <https://github.com/binyxu/VISTA/>

Abstract

Long-horizon tool agents are bottlenecked by how their context grows toward the limits of the context window. Recent systems make context management agent- or system-controlled, but they either learn compression policies that discard evidence or manage context in a layer the agent never sees. We argue that both miss a more basic gap: frontier language models are proprioceptively blind to their own context. From the prompt alone they cannot reliably infer block size, recency, or the remaining budget, all of which are needed for keep-or-archive decisions. We introduce VISTA (Visible Internal State for Tool Agents), a training-free, model-agnostic layer that represents working memory as typed addressable blocks, surfaces a runtime dashboard of token usage, recency, archive status, and remaining budget, and archives blocks as recoverable full-fidelity payloads. On LOCA-Bench, BrowseComp-Plus, and GAIA, the same untrained interface transfers across 1M-, 100K-, and 10K-scale trajectories. On LOCA-Bench it lifts Gemini-3-Flash from 22.7 to 50.7%, reaches 58.0% on BrowseComp-Plus, and remains competitive on GAIA. Gains grow with context pressure and transfer across backbones, while ablations confirm that the dashboard matters beyond archive and recovery tools.

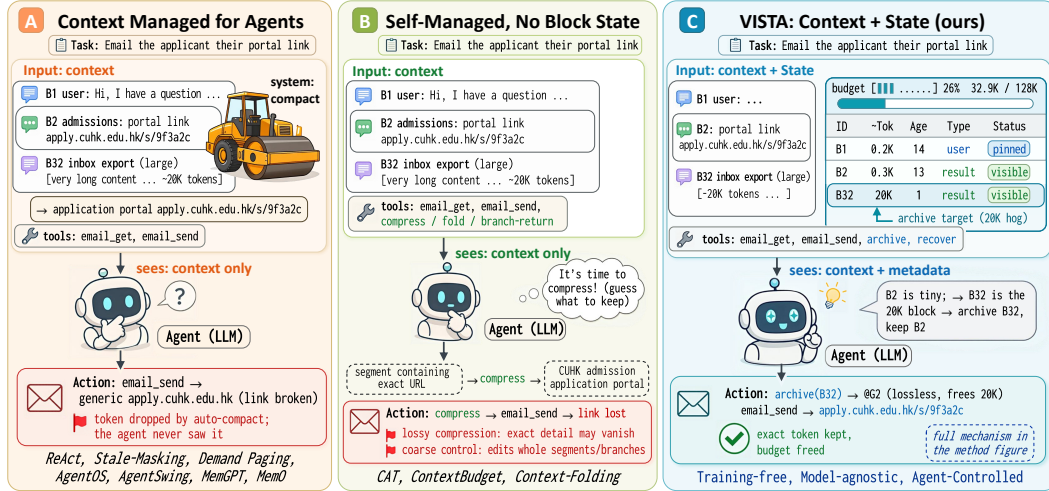


Figure 1: Who manages context, and on what information. Fixed rules compact context the agent cannot see, and blind self-management guesses without state. VISTA surfaces per-block metadata, so the agent archives the large block losslessly.

1 Introduction

Language agents operate over stateful tasks such as filling spreadsheets from web and email evidence, modifying databases, preparing application materials, debugging code, and coordinating business workflows [16, 34, 45]. Their context is working memory. It accumulates tool evidence, stale

observations, failed attempts, user constraints, hypotheses, file paths, and action contracts that must remain correct many steps later [22, 26, 31]. As the task runs, working memory grows until it crowds or overflows the context window, a pressure also studied in long reasoning systems that summarize or carry state across computation [2, 11, 29]. The agent must decide what to keep visible, what to set aside, and what to recover. How this growing context is managed determines whether long-horizon agents succeed.

Existing approaches differ in who makes these decisions. One family keeps the decision outside the agent. Stale-observation masking hides old tool outputs by rule [46], and OS-style layers page or evict context beneath the agent. These layers track statistics such as size, age, and usage, but only inside the runtime. The agent cannot inspect them, and a fixed rule cannot know which evidence will matter later. A second family moves the decision into the agent and learns it from data. Context-as-a-tool fine-tunes a compressor, and budget-aware methods train compression policies with reinforcement learning [16, 25, 26, 31]. These methods can improve performance, but they often discard evidence through summarization or deletion and are tied to the training setting. Across both families, the agent can read block contents but remains context-state blind: the prompt omits the runtime state needed for a keep-or-archive decision, including block size, recency, archive status, and remaining budget. Figure 1 contrasts these families with our approach on one task.

We take an elicitation view. We hypothesize that capable models already contain context-management competence from pretraining on note-taking, retrieval, and reorganization traces, and that a missing interface, not a missing policy, is the bottleneck. Context management is a meta-tool decision over the agent’s own working memory, made under partial observability. The agent must choose what to keep or externalize while the prompt omits the runtime state that governs the choice. Learned policies or distilled agent skills can compensate through training [36], but this entangles what information should be exposed with what policy should act on it. Figure 2 makes this motivation concrete: RL-based self-management implicitly improves both task performance and the model’s ability to recognize when its context needs management, whereas supplying the state dashboard makes that judgment easy even before adaptation. We therefore make the state explicit and express when-to-manage criteria as a rubric-style instruction. VISTA elicits these latent capabilities without requiring training or model-specific adaptation, while remaining compatible with RL.

This proprioceptive view implies three requirements. The interface must expose per-block token cost, recency, archive status, and remaining budget. It must be reversible, because one-way deletion or summarization can remove evidence needed later. It must be model-agnostic, so gains reflect elicitation through the interface, not training for one backbone or domain.

We introduce VISTA (Visible Internal State for Tool Agents), a context layer that represents working memory as typed, addressable blocks and surfaces a dashboard with per-block token usage, recency, archive status, and budget. The dashboard is a proprioceptive view of the agent’s context state. The agent can archive bulky blocks as external payloads with stable handles and recover exact bytes on demand. Archived payloads are exact transcripts, so removing a block from the prompt does not destroy it. We prove that both underlying resources are necessary: recoverability, because discarded evidence cannot otherwise be restored; and sufficiently informative proprioceptive state, because a size-blind manager over-archives or under-recovers even given recovery (Proposition 1 and Theorem 1). VISTA realizes both resources in one agent-facing interface. VISTA requires no training and wraps any backbone.

Empirically, Table 1 tests VISTA across million-, 100K-, and 10K-token trajectories in LOCA-Bench, BrowseComp-Plus, and GAIA. In LOCA-Bench [45], it solves 38/75 tasks versus 17 for ReAct and 32 for Claude Code, with lower trajectory cost than Claude Code. On BrowseComp-Plus, it reaches 58.0% versus 52.0% for the strongest baseline, and remains competitive on GAIA. Pressure sweeps

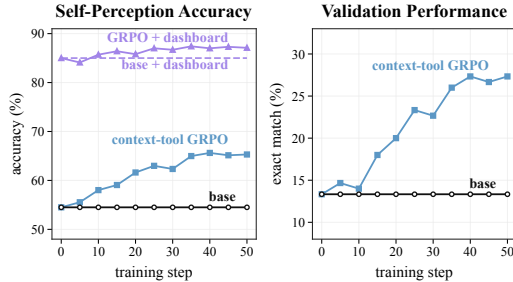


Figure 2: RL implicitly learns self-management. Across training, task performance and dashboard-free self-perception improve together, while dashboard-aided perception remains high. This gap suggests that the missing ingredient is observable state, motivating us to expose it explicitly instead of asking the policy to learn it implicitly.

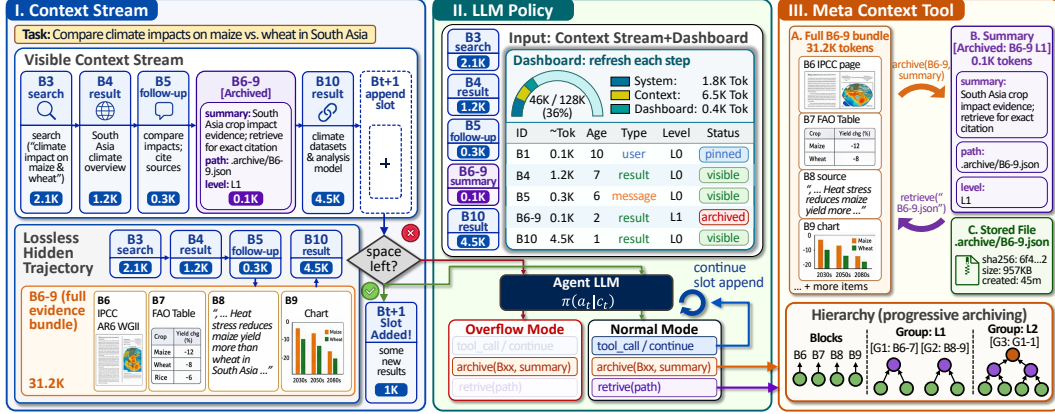


Figure 3: VISTA architecture. Messages and tool outputs become addressable blocks. The dashboard exposes budget and handles to the agent, while archived payloads remain recoverable outside the active prompt.

show that the advantage grows on long trajectories, where VISTA cuts active-context overhead while improving accuracy. The same untrained layer improves all four tested backbones, and ablations show that the dashboard matters beyond archive and recovery tools. In transfer, VISTA also reaches higher F1 than the specialized AMA-Bench agent without memory tuning.

This paper makes three contributions.

- We frame context management as a meta-tool decision under partial observability and identify context proprioception as the missing interface: LLM agents are context-state blind because their prompts omit reliable runtime state.
- We introduce VISTA, a training-free context layer whose dashboard exposes per-block metadata and pairs it with lossless archive and recovery. Two matched separations prove that recoverability and sufficiently informative proprioceptive state are jointly necessary; VISTA supplies both in a single agent-facing interface.
- We show across LOCA-Bench, BrowseComp-Plus, and GAIA that VISTA elicits self-management across trajectory scales and backbones, isolate the dashboard with ablations, and demonstrate transfer on AMA-Bench.

2 Methodology

VISTA treats context management as a meta-tool decision over the agent’s own working memory. Figure 3 shows the three-stage loop: a context stream, a refreshed dashboard, and archive/recovery tools. The goal is to make context state perceptible to an unmodified model without fine-tuning, model-specific changes, or destroyed evidence.

2.1 Problem Setup

At step t , a tool agent has a task goal g , raw interaction history H_t , environment tools $\mathcal{T}_{\text{env}}$, and a context budget B . We write the history as action-observation pairs $H_t = (a_1, o_1, \dots, a_{t-1}, o_{t-1})$. A standard ReAct-style harness serializes this append-only history into the next model input. Once the serialized history exceeds B , the harness must truncate, clear, mask, or summarize prior content. These interventions conflate what remains visible, what is preserved exactly, and what can be recovered later.

VISTA separates these choices through a workspace $W_t = (V_t, A_t)$ of visible blocks and recoverable archived payloads. The model does not act on the raw transcript directly; it acts on the workspace rendering

$$\begin{aligned} \tilde{C}_t &= \text{render}(V_t) \oplus \text{handles}(A_t) \oplus D_t, \\ C_t &= \text{preflight}(\tilde{C}_t, B), \quad a_t \sim \pi_\theta(\cdot | C_t), \quad |C_t| \leq B. \end{aligned} \quad (1)$$

Here D_t is the dashboard: a factual ledger of block IDs, token estimates, recency, block type, archive level, status, and remaining budget. The candidate $\tilde{C}_t$ may exceed the budget. In that case, preflight emits a budget-safe management rendering C_t that retains the dashboard, actionable handles, and compact stubs while disabling ordinary task tools; otherwise it returns the ordinary rendering. The raw transcript is still logged for evaluation, but the agent’s working memory is the workspace, not the append-only history.

Workspace invariants. The harness enforces a budget constraint, $|C_t| \leq B$, after preflight and final assembly; every actionable unit must have a stable block ID or handle; and archived payload bytes remain recoverable unless the agent explicitly deletes them. The dashboard reports the same token estimates used by these checks, so the agent sees the state that the harness will enforce.

2.2 Context Stream

The left panel of Figure 3 shows how VISTA rewrites the transcript as a block stream. The agent-facing state has two storage regimes:

- (i) **VISIBLE** blocks appear in the active prompt with exact content; pinned blocks are a protocol-required subset of this state.
- (ii) **ARCHIVED** blocks are replaced by compact handles and summaries, while their original bytes are stored externally.

Deletion is an explicit agent action, not a storage regime: it removes the selected content permanently. A separate hard-budget guard may replace already-visible raw tool results with recoverable offloaded placeholders; a new tool result that still cannot fit is rejected with a compact notice and can be rerun after context cleanup or with a narrower query. These safeguards are distinct from the agent’s archive decision.

This stream gives the agent an address space for context decisions. When a bundle of evidence becomes too large, the stream keeps a compact handle while the exact payload remains in the hidden trajectory. New observations are admitted only when the resulting request remains budget-safe.

Structure preservation. The visible stream is the next-call working set, while the hidden trajectory stores exact payloads that may be needed later. Tool results remain linked to their assistant tool calls, and archived results are rendered in protocol-valid form. If a parent call is also archived, the placeholder becomes ordinary context instead of an orphaned tool response.

2.3 LLM Policy

Dashboard input. The middle panel shows what the model receives: the visible context stream plus a dashboard. The dashboard is regenerated after every tool result is registered, so the agent acts on the current workspace state. It is a ledger over blocks, not a memory oracle: it exposes runtime state created by the harness or by the agent’s earlier actions and does not add hidden task evidence.

Unified action space. The same model policy chooses ordinary environment actions and context actions:

$$\begin{aligned} \mathcal{T}_{\text{ctx}} &= \{\text{archive}(\mathcal{S}, \rho), \text{delete}(\mathcal{S})\}, \\ a_t &\in \mathcal{T}_{\text{env}} \cup \mathcal{T}_{\text{ctx}} \cup \{\text{answer}\}. \end{aligned} \quad (2)$$

Here $\mathcal{T}_{\text{ctx}}$ contains the two dedicated workspace tools. Recovery uses an ordinary file or terminal read from the archive path and is therefore an environment action, which we denote abstractly by $\text{read_path}(h, q) \in \mathcal{T}_{\text{env}}$. Thus context management is not a separate post-hoc controller. It is part of the model’s action space, conditioned on the dashboard and task evidence still visible in C_t .

Mode switch. The dashboard is the proprioceptive channel of the meta-tool. From prompt text alone, a model cannot reliably infer how costly a block is, how recent it is, or whether it has been used again. These signals determine whether a block should stay visible, be archived, or be recovered. In normal mode, the agent may call environment tools, continue the task, archive blocks, or read archived payloads. In overflow mode, ordinary tool calls are disabled until the agent reduces the visible context. The allowed action set is therefore

$$\mathcal{A}_t = \begin{cases} \mathcal{T}_{\text{env}} \cup \mathcal{T}_{\text{ctx}} \cup \{\text{answer}\}, & |\tilde{C}_t| \leq B, \\ \mathcal{T}_{\text{ctx}}, & |\tilde{C}_t| > B. \end{cases} \quad (3)$$

The hard budget is enforced by the harness, but the agent chooses what to move. A final preflight guard can offload already-visible raw tool-result blocks near the hard limit; if a newly returned result still cannot be admitted, the harness rejects that result instead of silently truncating it. The complete loop is given in Appendix F.1.

2.4 Meta Context Tool

Archive interface. The right panel shows the meta-context tool. Archiving takes a selected block set $\mathcal{S}$ and a short replacement summary ρ :

$$\begin{aligned} h &= \text{archive}(\mathcal{S}, \rho), \\ \text{read_path}(h, \emptyset) &\equiv \text{payload}(\mathcal{S}). \end{aligned} \quad (4)$$

The first line creates a compact handle in the visible stream; the second states the lossless contract using an ordinary environment read, with no third context tool. The exact payload moves to the external archive, while h keeps the path, level, size, and checksum metadata visible, leaving a natural boundary for external auditing [37]. The optional selector q requests an exact range or subset of that payload; $q = \emptyset$ requests the full payload. Recovery is performed through ordinary file or terminal access to the stored payload path. There is no task-specific retrieval oracle. If the payload is too large, the agent may read bounded chunks or rerun the original source tool with narrower arguments.

The stored payload is a transcript of what the model saw, not a guarantee that the source was complete. If a source response was paginated or truncated, the archive preserves that result exactly and leaves re-querying to the agent.

Hierarchical recovery. Archiving is hierarchical. A first archive level may group several raw blocks into a bundle, as B6-9 does in the figure. Later, groups can themselves be archived into coarser handles when context pressure grows. The visible stream therefore stores a small summary and retrieval guide, while the hidden trajectory stores the exact evidence. This is why VISTA differs from summarization. Summaries guide navigation, but they are not the only representation of the evidence.

Recovery follows the hierarchy in reverse. The agent may inspect a coarse handle, recover the payload, and then decide whether a narrower piece of evidence should return to active context. It need not reload a long transcript when one row or identifier is needed. It can recover the file, search or read a bounded part, and continue with a smaller block.

2.5 RL Adaptation

The interface above is training-free: an instruction-following model can use the dashboard and context tools without changing its parameters. We additionally consider a post-training branch that refines *when* and *how* the same policy invokes archive and recovery. This branch leaves the workspace, dashboard, and lossless tool contract unchanged. Following the trajectory-level optimization pattern of Context-Folding [25], we optimize complete interaction trajectories while conditioning every model token on the workspace that was actually visible when that token was generated.

Workspace-consistent rollouts. For a training task g , the old policy samples a group of G trajectories $\{\tau_i\}_{i=1}^G$. Let $W_{i,t}$ be the workspace immediately before token $x_{i,t}$ and let $F(W_{i,t})$ denote the rendering in Eq. (1). Archive actions remove their selected payloads from subsequent renderings, whereas recovery actions restore only the content actually read. Let $H_{i,t} := F(W_{i,t}) \oplus x_{i,<t}^{\text{turn}}$ include the autoregressive prefix already generated in the current LLM turn. The importance ratio is therefore

$$r_{i,t}(\theta) = \frac{\pi_\theta(x_{i,t} \mid H_{i,t})}{\pi_{\text{old}}(x_{i,t} \mid H_{i,t})}, \quad (5)$$

and $m_{i,t} = \mathbf{1}_{\text{LLM}}(x_{i,t})$ masks tool observations. Thus rollout and policy optimization use the same managed context; reconstructing the raw, append-only transcript for the actor update would train a different policy.

Outcome reward and process penalties. Each trajectory receives a task reward $R_i \in [0, 1]$ from the benchmark scorer. Sparse task reward determines whether management preserved useful evidence. We add only penalties for objective workspace failures; we do not reward archive frequency or compression directly. Let $p_{i,t} = |F(W_{i,t})|/B$ be active-context pressure and ρ a soft pressure

threshold. The token-level process signal is

$$\begin{aligned} Q_{i,t} &= Q_{i,t}^{\text{pressure}} + Q_{i,t}^{\text{invalid}} + Q_{i,t}^{\text{undo}}, \\ Q_{i,t}^{\text{pressure}} &= -\lambda_p \max\left(0, \frac{p_{i,t} - \rho}{1 - \rho}\right). \end{aligned} \quad (6)$$

Pressure is computed once from the pre-action rendering—including visible blocks, handles, notices, and dashboard—and shared by all LLM tokens in that turn; a workspace transition affects pressure from the next turn onward. $Q_{i,t}^{\text{invalid}} = -\lambda_i$ iff the context-tool executor returns an error or the canonical workspace hash is unchanged, covering missing handles, malformed calls, and exact repetitions. $Q_{i,t}^{\text{undo}} = -\lambda_u$ marks an otherwise valid archive–recovery pair that restores the same workspace hash before any intervening environment observation or context growth; it is assigned post hoc to the LLM tokens of both action turns. The three terms add when they co-occur and are clipped only in Eq. (7). We use $\rho = 0.8$, $\lambda_p = \lambda_i = 1$, and $\lambda_u = 0.2$. Archiving below a fixed occupancy threshold is not penalized: a small prompt may still contain low-density content worth externalizing proactively.

We form an outcome-normalized, process-shaped token advantage

$$\hat{A}_{i,t} = \frac{\text{clip}(R_i + Q_{i,t}, 0, 1) - \mu_R}{\sigma_R + \epsilon}, \quad \mu_R = \frac{1}{G} \sum_{j=1}^G R_j, \quad (7)$$

where $\sigma_R := \sqrt{G^{-1} \sum_{j=1}^G (R_j - \mu_R)^2}$ is the within-task outcome standard deviation. Thus the group baseline and scale come from outcome reward, while $Q_{i,t}$ supplies local process shaping. The policy maximizes the clipped objective

$$J_{\text{RL}} = \mathbb{E} \left[\frac{1}{\sum_i |\tau_i|} \sum_{i,t} m_{i,t} \min\left(r_{i,t} \hat{A}_{i,t}, \text{clip}(r_{i,t}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t}\right) \right]. \quad (8)$$

Here $|\tau_i| := \sum_t m_{i,t}$ is the number of model-generated tokens in trajectory i . Only model-generated tokens contribute to Eq. (8); ordinary reasoning, context actions, recovery decisions, and final synthesis are optimized jointly. Training prompts are screened using training-split pilot rollouts to retain groups with both successful and unsuccessful samples, since all-equal groups yield no outcome-relative learning signal.

This reward deliberately excludes the number of context-tool calls, active-context reduction, and self-perception probe accuracy. Those quantities remain held-out mechanism measurements: optimizing them directly would permit trivial over-archiving and make any post-training perception gain circular.

2.6 Theory: Recovery and Proprioception Are Both Necessary

VISTA couples lossless recovery with an informative dashboard. The two resources solve different problems: recovery preserves evidence after eviction, whereas proprioception helps select what to evict. We formalize both with Fano’s inequality [23]: reconstruction gives the recovery separation, and list decoding gives the information–recovery tradeoff. We keep the main statements and intuition here; channel calculations, auxiliary lemmas, and full proofs are in Appendices A–D.

Recovery is necessary under budget pressure. In $\mathcal{T}_{N,k}$, the history contains N independent k -bit evidence blocks, but the prompt holds only $B < Nk$ bits. A uniformly random target block is revealed after compression and must be reproduced exactly. A non-recovering method retains only an in-prompt representation R with $H(R) \leq B$; a recovering method may reload the selected block after the reveal.

Proposition 1 (Recovery is necessary under budget pressure). *For any non-recovering method whose pre-reveal in-prompt state R satisfies $H(R) \leq B$,*

$$\Pr[\text{correct on } \mathcal{T}_{N,k}] \leq \frac{B}{Nk} + \frac{1}{k}.$$

VISTA is correct with probability 1 whenever the instruction, the N handles, and one recovered block fit within B . Thus there are feasible scaling regimes in which the non-recovering bound vanishes while VISTA stays at 1.

The bound captures delayed evidence demand: a lossy state cannot know which exact block will later matter, whereas archive handles preserve every block until the target is known.

Proprioception is necessary to use recovery efficiently. Recovery removes the losslessness barrier but not the *control* problem: the agent must still decide *which* blocks to move. We give both agents the same lossless archive and vary only how much state information identifies the best eviction target.

Definition 1 (Make-room instance $\mathcal{M}_{n,L,\ell}$). The working set holds n blocks. A hidden index J^* , uniform on $\{1, \dots, n\}$, marks one *bulky* block of size L ; the other $n - 1$ *load-bearing* blocks each have size $\ell < L$, with $\kappa := L/\ell \geq 2$ an integer. The prompt is over budget by L tokens. The bulky block is not needed again, but every load-bearing block is queried later; after each archive the agent observes only whether overflow remains. Sizes are net prompt tokens freed after inserting replacement handles.

To exit overflow the agent archives blocks in some order until the freed size reaches L . Let τ be the position at which J^* is archived: archiving J^* frees L at once, while each load-bearing block frees only ℓ , so κ of them are needed otherwise. Hence the number of load-bearing blocks that must later be made available again is

$$Z = \min(\tau - 1, \kappa). \quad (9)$$

Making room efficiently is therefore a localization problem: rank J^* early and avoid unnecessary recovery obligations.

Definition 2 (Proprioceptive interface of rate I). Before acting, the agent receives an observation Y and archives in some (Y, U) -measurable order, using private randomness $U \perp (J^*, Y)$. The *rate* of the interface is the information it carries about the bulky block, $I := I(J^*; Y)$ bits. A full size ledger identifies J^* and has $I = \log_2 n$. A dashboard-free endpoint observes content-derived noisy size signals; Proposition 2 in Appendix A bounds its induced rate. The theorem below otherwise applies to any observation Y through $I(J^*; Y)$.

Theorem 1 (Information–recovery tradeoff). Assume $2 \leq \kappa < n/2$ and let $\nu := \log_2 \frac{n-\kappa}{\kappa} > 0$. On $\mathcal{M}_{n,L,\ell}$ under an interface of rate I (Def. 2), the expected number Z of archived load-bearing blocks obeys

$$\mathbb{E}[Z] \geq \kappa \left[1 - \frac{I+1}{\nu} \right]_+.$$

Here $[x]_+ := \max\{x, 0\}$. Any policy that answers every later query correctly must make all Z of these blocks available again and therefore incurs at least $\mathbb{E}[Z]$ expected block-level recovery obligations. This counts blocks that must be restored, not file-read calls: one grouped payload read may discharge several obligations at once. The full ledger identifies J^* and attains $Z = 0$. At the other endpoint, any low-rate interface satisfying $(I+1)/\nu \rightarrow 0$ incurs $\mathbb{E}[Z]/\kappa \rightarrow 1$. For any fixed $\delta \in (0, 1)$, $\mathbb{E}[Z] \leq \delta\kappa$ requires $I \geq (1 - \delta)\nu - 1$ bits, and the balanced-bucket construction in Appendix D gives an order-matching upper bound.

Thus recoverability prevents irreversible evidence loss, while state information reduces unnecessary eviction and later restoration. The operational signature matches the ablations: without the ledger the agent archives far more and retrieves less (255/57 versus 69/105 archive/retrieve events; Figure 8b). The content-channel calculation, finite-sample corollaries, and achievability construction are deferred to the appendix.

3 Experiments

3.1 Experiment Setup

Benchmarks. We evaluate across three online regimes with different trajectory scales: LOCA-Bench [45] as the primary million-token stress test, BrowseComp-Plus [3] as a 100K-scale deep-research retrieval transfer, and GAIA [21] on a fixed 165-question validation subset as a shorter general-assistant setting. We additionally use AMA-Bench as a long-memory generalization benchmark: completed agent histories are replayed through the VISTA workspace before question answering, testing whether the same mechanism can operate as trajectory memory. LOCA-Bench is external to this work; we adopt its public 75-configuration suite and scoring protocol, counting errors and timeouts as failures. Full benchmark protocols, subsets, scoring rules, and budget settings are in the evaluation-details appendix. We keep comparisons within each benchmark and do not aggregate across heterogeneous score scales [38].

Method	Mechanism			LOCA-Bench		BrowseComp-Plus		GAIA	
	State	Ctrl	Recov	Acc↑	Traj↓	Acc↑	Traj↓	Acc↑	Traj↓
<i>Fixed external policy</i>									
ReAct	✗	✗	✗	22.7	3.51M	39.3	163K	61.2	23K
Tool-result Clearing	✗	✗	✗	26.7	2.60M	42.7	161K	65.5	24K
Stale-obs. Masking	✗	✗	✗	28.0	3.32M	38.0	112K	61.8	24K
Skeleton Compression	✗	✗	✗	33.3	2.84M	40.0	139K	70.3	28K
<i>Agent-mediated / lossy</i>									
SLIM (summary)	✗	~	✗	29.3	3.76M	49.3	162K	67.9	30K
Active Ctx. Compression	✗	✓	✗	36.0	3.20M	42.7	162K	71.5	39K
Context-Folding	✗	✓	✗	34.7	3.41M	43.3	166K	64.8	39K
<i>Lossless external store</i>									
Auto-Archive + Recover	✗	✗	✓	44.0	2.73M	45.3	133K	63.6	20K
Claude Code	✗	✓	~	42.7	6.72M	<u>52.0</u>	247K	73.9	44K
<i>Ours and ablations</i>									
VISTA w/o dashboard	✗	✓	✓	37.3	5.25M	50.0	423K	68.5	24K
VISTA w/o recovery	✓	✓	✗	<u>45.3</u>	2.99M	43.3	161K	72.1	28K
VISTA (full)	✓	✓	✓	50.7	2.86M	58.0	135K	<u>73.3</u>	33K

Table 1: Main results across scales. The three benchmarks span million-, 100K-, and 10K-token trajectories, testing whether context management transfers across operating regimes. The pattern supports our central claim: long-horizon gains require not just compression or storage, but an agent-visible state interface paired with controllable, lossless recovery.

Baselines and configuration. We compare against fixed external policies, agent-mediated compression, and production-agent baselines. The fixed-policy group includes ReAct, Tool-result Clearing, and stale-observation masking [46]. The agent-mediated group includes SLIM [42], Active Context Compression [26], and a structured Skeleton Compression baseline following context-as-a-tool compressors [16]. We also include Context-Folding [25], Auto-Archive + Recover, and Claude Code (CLI release May 6, 2026). On AMA-Bench, we compare with the benchmark AMA agent and retrieval-style memory baselines. Appendix Table 6 and the implementation appendix give the full capability matrix, prompts, flags, dashboard format, and tool definitions. Learned context managers whose released artifacts do not match this setting are discussed as complementary systems and excluded from direct empirical rankings.

VISTA is training-free and uses the same strategy across backbones. On LOCA-Bench, the main runs use Gemini-3-Flash with a 128K budget. At each turn, the agent sees per-block context metadata and may archive or recover exact transcript payloads; the task tools are unchanged. Across all LOCA-Bench comparisons we hold fixed the task instances, benchmark-native tools, backbone, context budget, and scoring. For harness-native baselines, the agent loop and prompt assembly are also fixed and only the context-management policy changes. Claude Code retains its complete released CLI harness, with its MCP task tools replaced by the benchmark-native equivalents. SLIM and Active Context Compression are reproduced as training-free inference-time baselines, while Skeleton Compression is a structured compression baseline inspired by context-as-a-tool compressors; it is not a trained CAT policy. VISTA uses no task-specific retrieval oracle: archived payloads are stored as exact transcripts, and recovery is performed through ordinary file or terminal reads from the returned archive path.

3.2 Main Results Across Scales

Table 1 compares VISTA across million-token LOCA-Bench trajectories, 100K-scale BrowseComp-Plus retrieval, and shorter GAIA trajectories. On LOCA-Bench, VISTA solves 50.7% of tasks, versus 22.7% for ReAct and 42.7% for Claude Code, while using less trajectory than Claude Code. On BrowseComp-Plus it reaches 58.0%, above the strongest baseline at 52.0%. On GAIA it remains competitive in the shorter setting, reaching 73.3% versus 73.9% for Claude Code. The mechanism columns separate state visibility, decision maker, and recovery; methods missing one of these pieces do not match the full interface consistently. The gain is largest in the settings where evidence must survive long trajectories: LOCA-Bench stresses repeated tool interaction, while BrowseComp-Plus

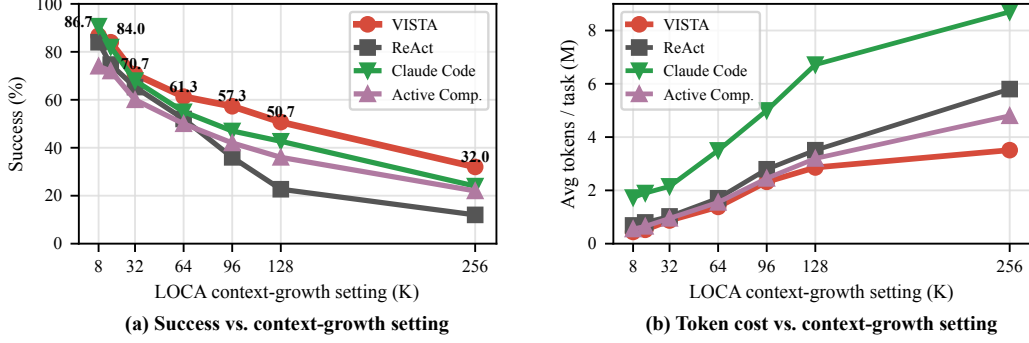


Figure 4: Pressure sweep. Across 8K–256K context growth, VISTA degrades more gracefully than ReAct; the right panel reports average API tokens per task.

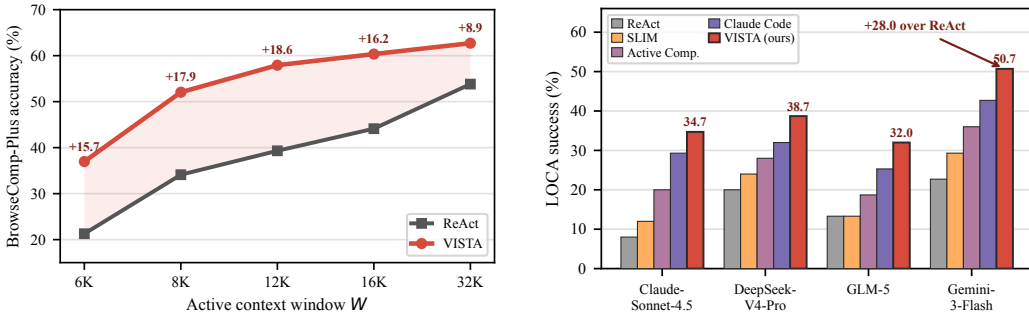


Figure 5: BrowseComp-Plus window sweep. With task set fixed, both methods improve as the active window grows, while VISTA’s gain over ReAct is largest at intermediate windows.

Figure 6: Cross-backbone results. The same untrained VISTA layer is best on all four backbones under the same benchmark task and scoring protocol.

stresses retrieval followed by delayed synthesis. The LOCA-Bench cost ledger also rules out a spend-more explanation: VISTA uses 2.86M tokens and 36.4 steps per task, compared with 6.72M tokens and 171.5 steps for Claude Code (Appendix Table 7).

Cache-aware dollar cost. Shorter trajectories need not be cheaper if context edits invalidate the reusable KV prefix. We therefore combine the token ledger with standard Gemini-3-Flash pricing: \$0.50/M uncached input, \$0.05/M cached input, and \$3.00/M output including thinking [7]. Thus VISTA remains the least expensive at \$0.93 per task while achieving the highest accuracy, versus \$1.02 for Active Context Compression, \$1.05 for Claude Code, and \$1.26 for ReAct.

3.3 Pressure Regimes

LOCA-Bench million-token pressure. LOCA-Bench creates the failure mode VISTA targets: useful observations arrive early, bulky tool results accumulate, and the agent must still act correctly many steps later. Figure 4 sweeps 8K to 256K context growth on the full 75-task LOCA-Bench suite; exact counts are in Appendix Table 9. The methods are close at low pressure, but the gap opens as distractor volume grows: one-way truncation degrades while recoverable externalization remains usable. At 8K the methods are essentially tied (86.7 versus 84.0), but by 128K the gap is 50.7 versus 22.7, and VISTA also spends fewer average tokens (2.86M versus 3.51M). This is the expected signature of recoverable working memory.

Usage → Method ↓	Input (M)	Output (M)	Cache rate (%)	Cost (USD)
Active Comp.	3.158	0.042	48.3	1.02
VISTA	2.783	0.080	56.1	0.93
ReAct	3.476	0.031	36.5	1.26
Claude Code	7.138	0.156	92.9	1.05

Table 2: Cache-aware Gemini-3-Flash cost on LOCA-Bench. Input includes cached tokens; output includes thinking.

BrowseComp-Plus retrieval pressure. BrowseComp-Plus tests whether the interface helps outside the workflow-heavy LOCA-Bench setting. Here the bottleneck is whether early retrieved evidence survives until synthesis. We set $W=12K$ below the median first-retrieval depth of the gold document ($\approx 17K$ tokens), so ReAct often loses early evidence while VISTA continues under budget. This setting isolates context-window pressure: loose windows usually keep the decisive document visible, so the methods are much closer. Figure 5 varies only W : tiny windows make dashboard overhead costly, large windows let ReAct retain enough evidence, and the gain peaks in the middle. VISTA also uses less active context than ReAct, though more cumulative API tokens, because it survives longer and issues more retrieval rounds.

3.4 Backbone Robustness

The cross-backbone result asks whether the effect is tied to one model family. It is not: the same untrained VISTA layer improves Claude-Sonnet-4.5, DeepSeek-V4-Pro, GLM-5, and Gemini-3-Flash at 128K (Figure 6), including stronger backbones. This supports the elicitation view: capable models can use a context-management interface when runtime state is visible.

3.5 Implicit Competence and RL Adaptation

Table 3 asks whether explicit state and a rubric for when to manage merely replace learning, or also provide a better substrate for it. They do both. The interface addresses the observability bottleneck, while RL can still refine the management policy acting on the exposed state. Here In-Distribution (ID) trains and evaluates on BrowseComp-Plus (BC+), whereas Out-Of-Distribution (OOD) trains on GAIA and evaluates on BC+. In distribution, the zero-shot variant improves substantially over the Base model, while our RL variant improves further and exceeds our context-tool GRPO baseline. For these post-training runs, we use Qwen3-8B and pre-screen training prompts with pilot roll-outs to increase the proportion of groups with nonzero reward variance; validation and test examples are never used for this selection. The same ordering persists when training moves to GAIA and evaluation remains on BrowseComp-Plus: the GAIA-trained policy transfers across search tasks, with our method retaining the best result. Thus the gain does not require RL: zero-shot nearly matches OOD post-training before any large-scale training, which motivates its use in our main experiments. Explicit state and rubric-guided decisions can also be combined with GRPO, while OOD transfer shows that the improvement is not solely training-set fit.

Method	ID $\uparrow$ BC+ $\rightarrow$ BC+	OOD $\uparrow$ GAIA $\rightarrow$ BC+
	<i>Zero-shot baselines</i>	
Base	13.3	13.3
Ours (zero-shot)	20.0 (+6.7)	20.0 (+6.7)
	<i>RL adaptation</i>	
	Context-tool GRPO	Ours (RL)
	27.3 (+14.0)	17.3 (+4.0)
	31.3 (+18.0)	21.3 (+8.0)

Table 3: Post-training and zero-shot transfer. Our RL variant exceeds context-tool GRPO in both regimes. Zero-shot trails ID post-training but nearly matches OOD post-training, motivating its use in our main large-scale experiments. Green: gain over Base.

Figure 7 localizes the implicit behavior exposed by Figure 2. Hard examples use more context-tool calls than easy ones throughout training, and tool use becomes more active as validation reward and self-perception improve. The stratification indicates that the learned policy does not apply one fixed management rate: it allocates more context-management actions to harder tasks and fewer to easier ones. Their co-movement provides mechanism evidence for difficulty-adaptive management. Active-context reduction also rises and remains largest on hard tasks, showing that the learned actions materially change the prompt state and do more than merely increase search effort. Together, these curves show that RL sharpens a state-conditioned management policy: it changes both when the agent intervenes and how much active context it frees. The complementary zero-shot and adapted results separate the value of exposing state from the value of optimizing the policy that acts on it. The matched design isolates these effects under the same backbone and evaluation protocol.

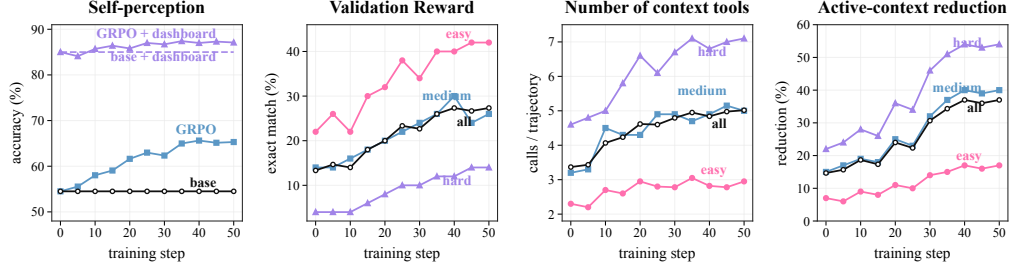


Figure 7: Context-tool GRPO learns when to manage. Dashboard-free perception and reward improve together, while dashboard-aided perception remains high. Harder tasks invoke more context tools and achieve larger active-context reductions, supporting difficulty-adaptive self-management.

3.6 Offline Trajectory-Memory Transfer

AMA-Bench removes live tool interaction but preserves the long-memory demand: the model must answer questions about completed agent histories. We adapt VISTA by replaying each trajectory into the workspace before QA, so actions and observations are organized into managed blocks instead of a flat prompt. This tests whether VISTA generalizes beyond on-line context control to a standard long-memory benchmark. On the full 208-episode comparison, VISTA leads on F1, stays within about two points of the specialized AMA agent on

judge accuracy, and does so at roughly a quarter of the per-episode runtime; it also outperforms BM25, EMem, and Mem0 adapters on F1. A training-free layer thus stays on par with a purpose-built memory agent while running far cheaper, a clean case of transfer to offline trajectory memory. Table 10 in Appendix gives the by-domain breakdown of AMA-Bench.

Method	F1	Acc.	Runtime/ep	Tokens/ep
BM25	0.335	0.575	35.5s	303.43K
EMem	0.363	0.651	166.2s	470.30K
Mem0	0.329	0.536	108.0s	30.63K
AMA	0.368	0.753	176.5s	268.98K
VISTA	0.382	0.731	43.7s	148.49K

Table 4: VISTA as replayed trajectory memory on AMA-Bench. Long-memory evaluation against the specialized AMA agent and memory-style adapters.

3.7 Mechanism Ablations

We probe the method along seven variants that hold the model, task set, token estimator, prompt assembly, and context limit fixed, changing only one mechanism at a time (Figure 8). Four of them remove a capability. No-archive drops recoverable externalization, no-dashboard removes the workspace map, no-recover hides payload paths, and fixed-archive replaces agent choice with a static rule. Two further variants leave every capability intact and perturb only the interface, one rephrasing the archive wording and one rendering the same state as a status board. Because capability and description move on separate axes, any remaining gap is attributable to the mechanism itself, not surface wording.

The two axes come apart cleanly. Removing a capability is costly, with no-archive falling to 27/75 and no-dashboard to 28/75, while the wording and status-board variants stay within a point or two of the full 38/75. The gain therefore follows the capability pathway, not phrasing. Tools alone are not enough either. No-dashboard keeps archive and recovery actions available, yet lacking the state to target them it issues 255 archives against only 57 recoveries, where the full system spends 69 archives and 105 recoveries (Figure 8b). This over-archive, under-retrieve pattern is blind offloading, not selective management, and it is precisely the behavior the rate-limited interface of Theorem 1 predicts. Taken together, these interventions identify a coupled mechanism: observability tells the agent when and what to externalize, while recoverability makes that externalization safe. Neither interface wording nor tool availability alone reproduces the full behavior, supporting our view of context management as closed-loop, state-aware control.

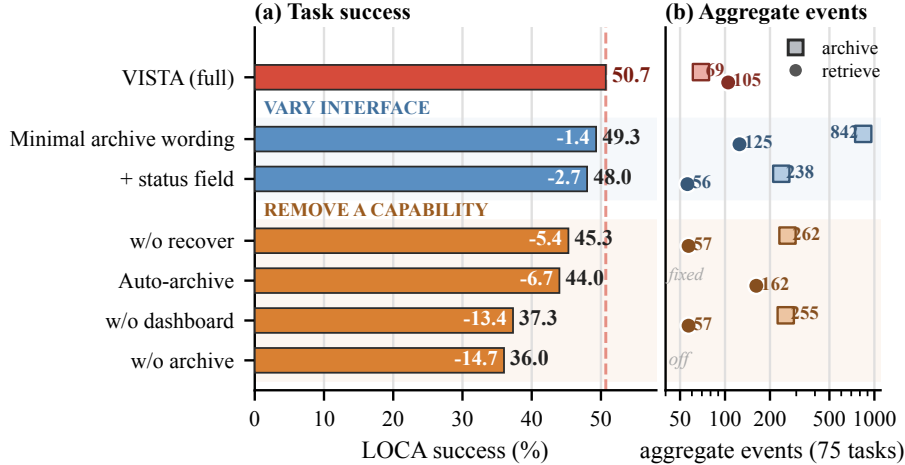


Figure 8: Component ablations. (a) Removing archive, dashboard, recovery, or agent choice hurts more than interface variants. (b) Aggregate archive vs. retrieve events over all 75 tasks (same grouping): without the dashboard the agent over-archives yet retrieves far less (255/57 vs. 69/105 for the full system), showing blind, nonselective archiving.

4 Analysis

Does the perception gap actually exist? VISTA assumes an agent is not given reliable runtime metadata about its own context. We directly test the token-magnitude component at the first archive moment of real LOCA-Bench runs. We strip the dashboard and ask the backbone to report its own state along three independent probes, namely total transcript size, individual block size, and pairwise size comparison, scoring each against exact token counts across four open and closed

Backbone	Total size		Block size		Pairwise	
	−dash	+dash	−dash	+dash	−dash	+dash
Claude-Sonnet-4.5	0.84	0.00	0.37	0.02	0.67	0.83
DeepSeek-V4-Pro	0.44	0.00	0.28	0.00	0.75	1.00
GLM-5	0.48	0.00	0.35	0.00	0.73	0.88
Gemini-3-Flash	0.43	0.00	0.24	0.00	0.68	1.00

Table 5: The perception gap is real and token-magnitude specific. Self-estimated context state with the dashboard stripped (−dash) versus present (+dash). Size is median relative error; pairwise is within-2× accuracy.

backbones (Table 5). The gap is consistent and large. Without the dashboard every backbone misjudges size, with median relative error from 0.43 to 0.84 and estimates essentially uncorrelated with truth. Adding the factual ledger reduces total-size error to zero on all four models and block-size error to 0–0.02, while lifting pairwise discrimination toward perfect. The effect is specific to token magnitude, not transcript memory, and it holds on open and closed weights alike, so the intervention is a factual interface, not a stronger prompt or a larger model. This is the empirical counterpart of the proprioceptive channel in Theorem 1, since the dashboard is what moves the agent from the size-blind regime toward accurate size observability. Together, these results establish context-state blindness in the dimension that directly governs budgeted context management: token magnitude. VISTA exposes recency and archive status alongside this measured signal.

What does a rescued run look like? Figure 9 shows one 128K LOCA-Bench email-triage run. The dashboard marks a large inbox-export block as the biggest item, the agent archives it, keeps the live prompt below the no-archive counterfactual, and later reads the exact payload back for the final action. A matched baseline summarizes or clears the block and cannot restore the verbatim value. Across rescued tasks, this full archive-then-recover loop appears in 8 of the 16 cases where VISTA archives at least one block. In 13 other archive-containing runs outside the rescued subset, archiving mainly frees space without later recovery. Thus VISTA does not win by discarding old evidence; it moves evidence out of view while preserving an addressable recovery path. The advantage is clearest on long trajectories, where VISTA lowers active-context overhead while improving accuracy.

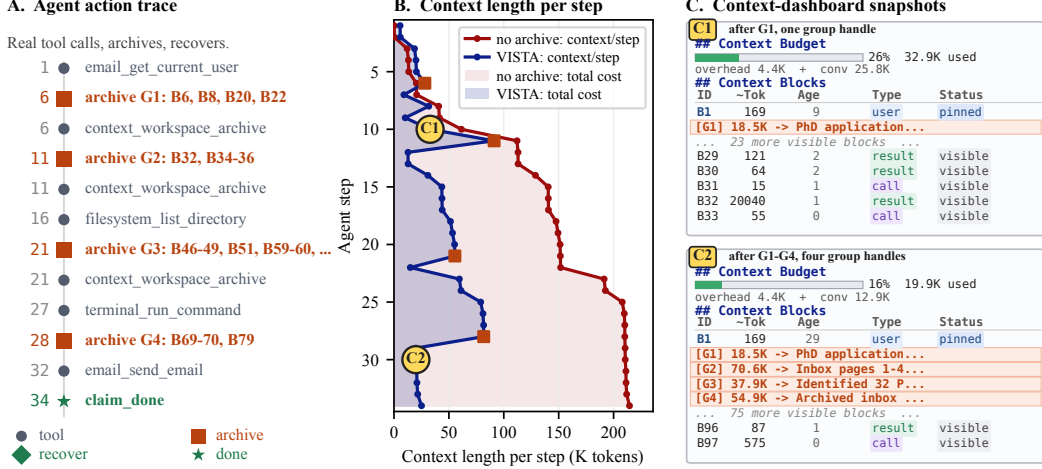


Figure 9: Case study trace. In one 128K LOCA-Bench run, VISTA archives large evidence, keeps the live context compact relative to a no-archive counterfactual, and recovers payloads when needed.

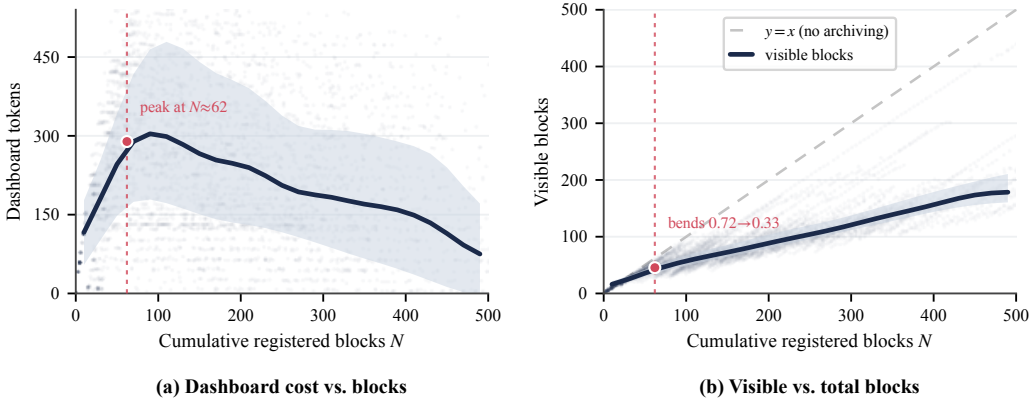


Figure 10: Dashboard overhead remains small across the observed trajectory range. (a) Net dashboard tokens (tool schemas and system prompt excluded) against cumulative registered blocks N : the cost peaks near $N \approx 62$ and then declines over the measured range. (b) The visible working set grows sublinearly in N (slope bends $0.72 \rightarrow 0.33$ at the same N). Solid lines are binned medians; shaded bands are interquartile ranges.

Does the dashboard scale? The dashboard adds tokens, so a fair worry is how it grows as a trajectory registers more blocks. We measure dashboard cost on three $n=30$ BrowseComp-Plus runs drawn from the official $N=150$ evaluation (deepseek-v4-pro, $W=12,288$, $B=163,840$). These runs are used only for the overhead trace; the reported 58.0% accuracy is computed on the full $N=150$ evaluation. We use the harness’s per-turn accounting over all 30 queries per run. Each turn logs a fixed overhead of tool schemas plus system prompt (a constant 492 tokens across every query, computed with the c1100k_base tokenizer) and the dashboard message; we subtract the fixed component to isolate the dashboard’s own cost. Figure 10(a) shows that this net footprint is not linear in the trajectory length: it peaks near $N \approx 62$ at roughly 340 tokens and then *declines* as N grows over the observed range. Figure 10(b) shows that the visible set also grows sublinearly, with its fitted slope bending from 0.72 to 0.33 near $N \approx 62$. Over the evaluated range, the dashboard footprint remains small and decouples from total trajectory growth.

5 Related Work

Prior work clarifies where context decisions sit and what state the agent can observe.

Context managed for the agent. One line keeps context decisions outside the model policy. Stale-observation masking, Demand Paging, AgentOS, and AgentSwing hide, page, or route context in the runtime [6, 12, 18, 46]; such systems may track size, age, or usage, but that state remains internal to the controller. Structured-eviction and cache-efficient managers such as Context Window Lifecycle and TokenPilot push this line further, replacing lossy summarization with deterministic, semantically aware pruning and cache-friendly ingestion to extend the working horizon at lower token cost [24, 40], yet they still decide what to evict on the agent’s behalf. Memory and retrieval systems such as MemGPT, Mem0, SimpleMem, MR.Agent, PlugMem, BudgetMem, and SkillPro organize prior experience through virtual context, long-term memory, or retrieval/graph stores [5, 9, 15, 20, 22, 41, 47]. These systems provide useful storage or routing substrates, but the agent does not receive a per-block map of its active prompt. In Figure 1, this corresponds to context being managed *for* the agent: the runtime may discard stale observations, page records, or retrieve memories, but the model sees only the resulting prompt. It therefore cannot directly decide which visible evidence should stay, move out of view, or return later, nor can it trade a small but critical item against a large expendable tool result.

Self-managed context without state metadata. A second line makes context management part of the agent loop. Context as a Tool, Active Context Compression, ContextBudget, Context-Folding, and LongSeeker expose compression, folding, deletion, or routing actions, often through learned policies or specialized controllers [16, 17, 25, 26, 31]. This line is closest to our notion of self-management: the agent or controller can invoke context operations instead of passively waiting for truncation. However, these operations usually act on summaries, milestones, commits, or branches, not on a persistent block table. ContextBudget makes token pressure explicit, and Context-Folding gives branch and return actions, but they do not expose the VISTA-style state needed to target a precise oversized block while preserving a small exact detail. Other systems add hierarchy, compression guidance, agent-compatible managers, learned memory operations, or compact reasoning summaries [2, 8, 10, 11, 13, 27, 29, 30, 43, 44, 48], and PACE adapts memory granularity by predicting each item’s relevance to the next action [28]. These works support our premise that context management is an agent-level decision, but their compressed representations are generally lossy and coarse relative to tool traces that contain exact identifiers, URLs, or table rows. VISTA instead isolates what runtime state the agent must perceive, then pairs that perception with block-level lossless archive and recovery.

Self-state awareness. Work on budget awareness, temporal blindness, and agent externalization studies signals that models cannot infer from prompt contents alone [1, 4, 14, 50]. Complementary probing work finds that agent-critical information such as plans resides in context and does not persist in hidden state, decaying once removed from the visible context [19], motivating an explicit self-description of working memory instead of assuming the model retains it. VISTA treats context state as such a signal and supplies it externally at inference time. This framing separates our contribution from simply enlarging the context window or improving summarizers: the missing information is not only more text, but a compact self-description of the agent’s current working memory. In our setting, that self-description includes which blocks exist, how large and old they are, whether they are visible or archived, and whether they can be recovered exactly. This is the Figure 1 contrast: the agent does not merely see context, but also sees a compact state view over that context. We evaluate this view mainly on LOCA-Bench, which stresses online tool agents under controllable context growth [45], and use AMA-Bench as a memory-oriented transfer point [49].

6 Conclusion

We identified context-state blindness as a basic bottleneck in long-horizon agents: the prompt omits an explicit, reliable view of the runtime state governing working memory. Our diagnostic establishes severe token-magnitude blindness across four open and closed backbones, and our theory proves a sharp information threshold for using recoverable memory efficiently. VISTA addresses this with a runtime dashboard plus lossless archive and recovery, giving the agent context-state information it can act on without destroying evidence. With no training, the same interface outperforms ReAct, deletion, masking, compaction, and Claude Code on LOCA-Bench and transfers across four backbones. The result shows that context management is not only a policy-learning problem: latent capabilities can be elicited by making hidden runtime state perceptible, positioning the interface as a complement to post-training, not a replacement.

References

- [1] Christopher Ackerman. Evidence for limited metacognition in LLMs, 2026. URL <https://arxiv.org/abs/2509.21545>.
- [2] Milad Aghajohari, Kamran Chitsaz, Amirhossein Kazemnejad, Sarath Chandar, Alessandro Sordoni, Aaron Courville, and Siva Reddy. The markovian thinker: Architecture-agnostic linear scaling of reasoning. *arXiv preprint arXiv:2510.06557*, 2025.
- [3] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025. URL <https://arxiv.org/abs/2508.06600>.
- [4] Yize Cheng, Arshia Soltani Moakhar, Chenrui Fan, Parsa Hosseini, Kazem Faghieh, Zahra Soda-gar, Wenxiao Wang, and Soheil Feizi. Your LLM agents are temporally blind: The misalignment between tool use decisions and human time perception. *arXiv preprint arXiv:2510.23853*, 2025.
- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [6] Zhaopeng Feng, Liangcai Su, Zhen Zhang, Xinyu Wang, Xiaotian Zhang, Xiaobin Wang, Runnan Fang, Qi Zhang, Baixuan Li, Shihao Cai, et al. AgentSwing: Adaptive parallel context management routing for long-horizon web agents. *arXiv preprint arXiv:2603.27490*, 2026.
- [7] Google. Gemini developer api pricing. <https://ai.google.dev/gemini-api/docs/pricing>, 2026. Gemini 3 Flash Preview standard pricing; accessed July 31, 2026.
- [8] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32779–32798, 2025.
- [9] Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for llm agents, 2026. URL <https://arxiv.org/abs/2606.06036>.
- [10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [11] Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyang Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. Memento: Teaching llms to manage their own context. *arXiv preprint arXiv:2604.09852*, 2026.
- [12] ChengYou Li, XiaoDong Liu, XiangBao Meng, and XinYu Zhao. Architecting agentos: From token-level context to emergent system-level intelligence, 2026. URL <https://arxiv.org/abs/2602.20934>.
- [13] Jiaqing Liang, Jinyi Han, Weijia Li, Xinyi Wang, Zhoujia Zhang, Zishang Jiang, Ying Liao, Tingyun Li, Ying Huang, Hao Shen, et al. GenericAgent: A token-efficient self-evolving llm agent via contextual information density maximization. *arXiv preprint arXiv:2604.17091*, 2026.
- [14] Yuxiang Lin, Zihan Wang, Mengyang Liu, Yuxuan Shan, Longju Bai, Junyao Zhang, Xing Jin, Boshan Chen, Jinyan Su, Xingyao Wang, Jiabin Pei, and Manling Li. Bagen: Are llm agents budget-aware?, 2026. URL <https://arxiv.org/abs/2606.00198>.
- [15] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026.
- [16] Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon swe-agents. *arXiv preprint arXiv:2512.22087*, 2025.
- [17] Yijun Lu, Rui Ye, Yuwen Du, Jiajun Wang, Songhua Liu, and Siheng Chen. Longseeker: Elastic context orchestration for long-horizon search agents. *arXiv preprint arXiv:2605.05191*, 2026.
- [18] Tony Mason. The missing memory hierarchy: Demand paging for llm context windows, 2026. URL <https://arxiv.org/abs/2603.09023>.

-
- [19] Aman Mehta and Anupam Datta. Plans don't persist: Why context management is load bearing for llm agents. *arXiv preprint arXiv:2606.22953*, 2026.
 - [20] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents, 2026. URL <https://arxiv.org/abs/2602.01869>.
 - [21] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general ai assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
 - [22] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
 - [23] Yuta Sakai. Generalizations of Fano's inequality for conditional information measures via majorization theory. *Entropy*, 22(3):288, 2020. doi: 10.3390/e22030288.
 - [24] Andrew Semenov and Svyatoslav Dorofeev. Beyond compaction: Structured context eviction for long-horizon agents. *arXiv preprint arXiv:2606.11213*, 2026.
 - [25] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.
 - [26] Nikhil Verma. Active context compression: Autonomous memory management in llm agents. *arXiv preprint arXiv:2601.07190*, 2026.
 - [27] Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. Memex (rl): Scaling long-horizon llm agents via indexed experience memory. *arXiv preprint arXiv:2603.04257*, 2026.
 - [28] Lei Wei, Xiao Peng, Guannan Zhang, Chenhao Jiang, Hongyu Li, Lanbo Lin, Yuanwu Xu, Jiayao Liu, Kesu Wang, Bin Wang, et al. Pace: Predictive adaptive context extraction for long-horizon llm agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27184–27199, 2026.
 - [29] Ian Wu, Yuxiao Qu, Amrith Setlur, and Aviral Kumar. Reasoning cache: Continual improvement over long horizons via short-horizon rl. *arXiv preprint arXiv:2602.03773*, 2026.
 - [30] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.
 - [31] Yong Wu, YanZhao Zheng, TianZe Xu, ZhenTao Zhang, YuanQiang Yu, JiHuai Zhu, Chao Ma, BinBin Lin, BaoHua Dong, HangCheng Zhu, et al. Contextbudget: Budget-aware context management for long-horizon search agents. *arXiv preprint arXiv:2604.01664*, 2026.
 - [32] Binyan Xu, Xilin Dai, Di Tang, and Kehuan Zhang. One surrogate to fool them all: Universal, transferable, and targeted adversarial attacks with clip. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 3087–3101, 2025.
 - [33] Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. Clip-guided backdoor defense through entropy-based poisoned dataset separation. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 7415–7423, 2025.
 - [34] Binyan Xu, Xilin Dai, Fan Yang, and Kehuan Zhang. When agent automation becomes profitable: Quantifying and insuring autonomous ai risk through trace-economic underwriting. *arXiv preprint arXiv:2606.16465*, 2026.
 - [35] Binyan Xu, Xilin Dai, and Kehuan Zhang. Contextual agentic memory is a memo, not true memory. *arXiv preprint arXiv:2604.27707*, 2026.
 - [36] Binyan Xu, Dong Fang, Haitao Li, and Kehuan Zhang. From multi-agent to single-agent: When is skill distillation beneficial? *arXiv preprint arXiv:2604.01608*, 2026.
 - [37] Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. From internal diagnosis to external auditing: A vlm-driven paradigm for data-free online backdoor defense. In *Forty-third International Conference on Machine Learning*, 2026.
 - [38] Binyan Xu, Fan Yang, Xilin Dai, and Kehuan Zhang. Reviewer scores are not comparable across research areas in ml peer review. *arXiv preprint arXiv:2607.27209*, 2026.

-
- [39] Binyan Xu, Fan Yang, Di Tang, Xilin Dai, and Kehuan Zhang. Breaking the stealth-potency trade-off in clean-image backdoors with generative trigger optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 27197–27205, 2026.
 - [40] Buqiang Xu, Zirui Xue, Dianmou Chen, Chenyang Fu, Chiyu Wu, Caiying Huang, Chen Jiang, Jizhan Fang, Xinle Deng, Yijun Chen, et al. Tokenpilot: Cache-efficient context management for llm agents. *arXiv preprint arXiv:2606.17016*, 2026.
 - [41] Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. PlugMem: A task-agnostic plugin memory module for llm agents, 2026. URL <https://arxiv.org/abs/2603.03296>.
 - [42] Howard Yen, Ashwin Paranjape, Mengzhou Xia, Thejas Venkatesh, Jack Hessel, Danqi Chen, and Yuhao Zhang. Lost in the maze: Overcoming context limitations in long-horizon agentic search. *arXiv preprint arXiv:2510.18939*, 2025.
 - [43] Lu Yi, Runlin Lei, Liuyi Yao, Yuexiang Xie, Yuyang Li, Wenhao Zhang, Zhewei Wei, Yaliang Li, and Jian-Yun Nie. Learning agent-compatible context management for long-horizon tasks. *arXiv preprint arXiv:2605.30785*, 2026.
 - [44] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.
 - [45] Weihao Zeng, Yuzhen Huang, and Junxian He. LOCA-bench: Benchmarking language agents under controllable and extreme context growth. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
 - [46] Haoxiang Zhang, Qixin Xu, Zhuofeng Li, Lei Zhang, Pengcheng Jiang, Yu Zhang, and Julian McAuley. Masking stale observations helps search agents—until it doesn’t: A regime map and its mechanism. *arXiv preprint arXiv:2606.00408*, 2026.
 - [47] Haozhen Zhang, Haodong Yue, Tao Feng, Quanyu Long, Jianzhu Bao, Bowen Jin, Weizhi Zhang, Xiao Li, Jiaxuan You, Chengwei Qin, and Wenya Wang. Learning query-aware budget-tier routing for runtime agent memory, 2026. URL <https://arxiv.org/abs/2602.06025>.
 - [48] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 19149–19164, 2026.
 - [49] Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, et al. AMA-bench: Evaluating long-horizon memory for agentic applications. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
 - [50] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL <https://arxiv.org/abs/2604.08224>.

A Technical Setup for the Information–Recovery Analysis

The main text states the two separations and their operational interpretation. Here we record the content-derived endpoint and the auxiliary list-decoding inequality used by Theorem 1.

For the dashboard-free endpoint of Definition 2, let the agent observe $W_j = \ln s_j + \varepsilon_j$ for every block, with independent $\varepsilon_j \sim \mathcal{N}(0, \sigma^2)$ and $\sigma > 0$. This is an analyzable model of noisy size perception, not an estimate fitted from the empirical probe in Appendix I.

Proposition 2 (Content-derived information is bounded for fixed size ratio). *For the channel above,*

$$I_{\text{content}} = I(J^*; W) \leq \frac{(\log_2 \kappa)^2}{\sigma^2} \ln 2 \quad \text{bits.}$$

Thus for fixed (κ, σ) , content-derived size information is bounded independently of the number of blocks n , whereas an exact size ledger can identify one block among n .

Lemma 1 (Fano, list-decoding form). *Let J^* be uniform on $\{1, \dots, n\}$ and let $\mathcal{L}(Y, U)$ be any (Y, U) -measurable list of size $1 \leq m \leq n - 1$, where $U \perp (J^*, Y)$. With $I = I(J^*; Y)$ and $P_c = \Pr[J^* \in \mathcal{L}]$,*

$$\log_2 n - I \leq 1 + P_c \log_2 m + (1 - P_c) \log_2(n - m).$$

B Proof of Proposition 1

This is Fano’s inequality in *reconstruction* mode: a budget-limited state cannot reproduce more blocks than it has bits for. We restate the setting. The history holds N blocks $X_1, \dots, X_N$, each an independent string of k uniformly random bits, so $H(X_i) = k$ and the X_i are mutually independent. A non-recovering method holds a pre-reveal in-prompt state R with $H(R) \leq B$. The state R is a function of the blocks and the method’s internal randomness, formed before the query index i^* is revealed, and i^* is drawn uniformly on $\{1, \dots, N\}$ independently of everything else. After the reveal the method outputs a guess $g(R, i^*)$, and it is correct when $g(R, i^*) = X_{i^*}$.

Let $P_e^{(i)} = \Pr[g(R, i) \neq X_i]$ and let the reported success probability be $1 - P_e$ with $P_e = \frac{1}{N} \sum_{i=1}^N P_e^{(i)}$, the average over the uniform i^* .

Step 1: per-block Fano bound. Fix a block i . Since X_i is uniform on an alphabet of size 2^k , Fano’s inequality applied to the estimator $g(R, i)$ gives

$$H(X_i | R) \leq H_b(P_e^{(i)}) + P_e^{(i)} \log_2(2^k - 1) \leq 1 + P_e^{(i)} k,$$

where H_b is the binary entropy function, bounded by 1.

Step 2: independence couples the blocks to a budget. Because the X_i are mutually independent, $H(X_{1:N}) = \sum_i H(X_i)$, and subadditivity of conditional entropy gives $H(X_{1:N} | R) \leq \sum_i H(X_i | R)$. Hence

$$\begin{aligned} \sum_{i=1}^N I(X_i; R) &= \sum_i (H(X_i) - H(X_i | R)) \\ &\leq H(X_{1:N}) - H(X_{1:N} | R) \\ &= I(X_{1:N}; R) \leq H(R) \leq B. \end{aligned}$$

Step 3: combine. Using $I(X_i; R) = k - H(X_i | R) \geq k - 1 - P_e^{(i)} k$ from Step 1 and summing,

$$\begin{aligned} B &\geq \sum_{i=1}^N I(X_i; R) \\ &\geq \sum_{i=1}^N (k - 1 - P_e^{(i)} k) = Nk - N - k \sum_{i=1}^N P_e^{(i)}. \end{aligned}$$

Dividing by Nk and using $P_e = \frac{1}{N} \sum_i P_e^{(i)}$,

$$P_e \geq 1 - \frac{1}{k} - \frac{B}{Nk},$$

$$\Pr[\text{correct}] = 1 - P_e \leq \frac{B}{Nk} + \frac{1}{k}.$$

VISTA attains probability one. VISTA writes each block to external storage as an exact transcript and keeps only a compact handle in the prompt. The pre-reveal prompt holds the instruction and N handles, whose size is $O(N \log N)$, not $O(Nk)$. Once i^* is revealed, the agent reads payload i^* and recovers X_{i^*} byte for byte. Whenever the instruction, the handles, and one recovered block fit within B , the method emits X_{i^*} exactly, so its success probability is 1.

Asymptotic separation. The gap statement requires a regime in which VISTA *stays feasible* while the lossy bound vanishes. Feasibility needs the N handles plus one recovered block to fit, i.e. $B \geq c_0 N \log_2 N + k$ for the constant c_0 set by the handle encoding; the lossy bound $\frac{B}{Nk} + \frac{1}{k}$ vanishes when $Nk/B \rightarrow \infty$ and $k \rightarrow \infty$. Both hold, for example, at $k = \lceil \sqrt{N} \rceil$ and $B = cN \log_2 N$ with $c > c_0$: then B dominates the handle cost, so VISTA is feasible and stays at success 1, while $\frac{B}{Nk} + \frac{1}{k} = \Theta(\frac{\log N}{\sqrt{N}}) \rightarrow 0$, so the lossy success probability tends to 0 and the gap tends to 1 as $N \rightarrow \infty$. A *fixed* B does not exhibit this separation, because then VISTA's own $O(N \log N)$ handle table eventually violates the budget. The separation is thus a statement about the growth rate of the raw evidence Nk relative to a budget B that grows only fast enough to index it. $\square$

C Proof of Proposition 2

We locate the dashboard-free endpoint of Def. 2 for $\sigma > 0$. Conditioned on $J^* = j$, the observation $W = (W_1, \dots, W_n)$ is Gaussian with independent coordinates, $W_i \sim \mathcal{N}(m_i^{(j)}, \sigma^2)$, where the mean vector $m^{(j)}$ has $m_j^{(j)} = \ln L$ (the bulky block) and $m_i^{(j)} = \ln \ell$ for $i \neq j$. Let P_j denote this conditional law and $\bar{P} = \frac{1}{n} \sum_k P_k$ the mixture.

Step 1: mutual information as mixture KL. Writing mutual information in bits and KL with natural logarithms, for J^* uniform a standard identity gives

$$I(J^*; W) = \frac{1}{n \ln 2} \sum_{j=1}^n \text{KL}(P_j \| \bar{P}).$$

Because $\text{KL}(P \| \cdot)$ is convex in its second argument and $\bar{P} = \frac{1}{n} \sum_k P_k$, Jensen gives $\text{KL}(P_j \| \bar{P}) \leq \frac{1}{n} \sum_k \text{KL}(P_j \| P_k)$, hence

$$I(J^*; W) \leq \frac{1}{n^2 \ln 2} \sum_{j=1}^n \sum_{k=1}^n \text{KL}(P_j \| P_k).$$

Step 2: pairwise Gaussian KL. P_j and P_k are Gaussians with the same covariance $\sigma^2 \mathbf{I}$, so $\text{KL}(P_j \| P_k) = \frac{1}{2\sigma^2} \|m^{(j)} - m^{(k)}\|_2^2$. For $j \neq k$ the mean difference is nonzero only in coordinates j and k : coordinate j contributes $\ln L - \ln \ell = \ln \kappa$ and coordinate k contributes $\ln \ell - \ln L = -\ln \kappa$, so $\|m^{(j)} - m^{(k)}\|_2^2 = 2(\ln \kappa)^2$ and

$$\text{KL}(P_j \| P_k) = \frac{(\ln \kappa)^2}{\sigma^2} \quad (j \neq k), \quad \text{KL}(P_j \| P_j) = 0.$$

Step 3: combine. There are $n(n-1)$ ordered pairs with $j \neq k$, so

$$I(J^*; W) \leq \frac{1}{n^2 \ln 2} n(n-1) \frac{(\ln \kappa)^2}{\sigma^2} \leq \frac{(\ln \kappa)^2}{\sigma^2 \ln 2} \text{ bits} = \frac{(\log_2 \kappa)^2}{\sigma^2} \ln 2 \text{ bits},$$

the last equality using $\ln \kappa = (\ln 2) \log_2 \kappa$. The bound depends only on the log size-ratio $\log_2 \kappa$ and the perception noise σ . For fixed κ and σ , it is constant in the workspace size n , while the full ledger carries $\log_2 n$ bits. More generally, Theorem 1 places this content channel on the costly side of the make-room threshold whenever $(I_{\text{content}} + 1)/\log_2(n/\kappa) \rightarrow 0$. $\square$

D Proof of Theorem 1

The proof has five parts: (A) the list-Fano lemma; (B) a reduction that turns the make-room recovery cost into a list-decoding error, converting the information budget I into a lower bound on $\mathbb{E}[Z]$; (C) the closed-form tradeoff and its corollaries (endpoints, threshold, price); (D) an order-wise achievability construction; and (E) the reduction from adaptive archive policies to ordered probing. Throughout, J^* is uniform on $[n] := \{1, \dots, n\}$, $\kappa = L/\ell$ is an integer satisfying $2 \leq \kappa < n/2$, and $\nu := \log_2 \frac{n-\kappa}{\kappa} > 0$.

(A) List-Fano lemma (proof of Lemma 1). Let $\mathcal{L} = \mathcal{L}(Y, U)$ be a list of size m and $E := \mathbf{1}[J^* \notin \mathcal{L}]$, so $\Pr[E=0] = P_c$. Expand $H(J^*, E \mid Y, U)$ two ways. Since E is a function of (J^*, Y, U) , $H(J^*, E \mid Y, U) = H(J^* \mid Y, U)$. Also

$$H(J^*, E \mid Y, U) = H(E \mid Y, U) + H(J^* \mid E, Y, U) \leq 1 + H(J^* \mid E, Y, U).$$

Condition on E : given $E=0$, J^* lies in a set of size $\leq m$, so $H(J^* \mid E=0, Y, U) \leq \log_2 m$; given $E=1$, J^* lies in the complement of size $\leq n - m$ (recall $\mathcal{L}$ is (Y, U) -measurable), so $H(J^* \mid E=1, Y, U) \leq \log_2(n - m)$. Averaging with weights $P_c, 1 - P_c$ and using $H(J^* \mid Y, U) = H(J^*) - I(J^*; Y, U) = \log_2 n - I$ (as $U \perp (J^*, Y)$, $I(J^*; Y, U) = I(J^*; Y) = I$),

$$\log_2 n - I \leq 1 + P_c \log_2 m + (1 - P_c) \log_2(n - m),$$

which is Lemma 1. $\square$

(B) Reduction: recovery cost lower-bounds via list decoding. By the “still-over” argument (part (E) below), it suffices to prove the lower bound for the more powerful class of sequential policies. Such a policy archives blocks in an order $\pi = (\pi_1, \pi_2, \dots)$ measurable in (Y, U) and stops at the first time the freed size reaches L . Because the bulky block frees L by itself while each load-bearing block frees only ℓ , the process stops exactly at step $T = \min(\tau, \kappa)$ where τ is the position of J^* in π : if $\tau \leq \kappa$ it captures J^* and frees L ; if $\tau > \kappa$ it has archived κ load-bearing blocks, freeing $\kappa\ell = L$, before reaching J^* . The number of archived load-bearing blocks, which are exactly the ones that must later be recovered, is (9), $Z = \min(\tau - 1, \kappa)$.

Fix any integer $1 \leq m \leq \kappa$ and let $\mathcal{L}_m := \{\pi_1, \dots, \pi_m\}$, the first m probed blocks; this is a size- m , (Y, U) -measurable list. The key observation is

$$Z \geq m \iff \tau > m \iff J^* \notin \mathcal{L}_m, \quad (10)$$

because $Z = \min(\tau - 1, \kappa) \geq m$ iff $\tau - 1 \geq m$ (using $m \leq \kappa$) iff J^* is not among the first m probes. Hence, writing $P_c(m) := \Pr[J^* \in \mathcal{L}_m]$, we have $\Pr[Z \geq m] = 1 - P_c(m)$.

Expectation via tail sum. Since $Z \in \{0, 1, \dots, \kappa\}$,

$$\mathbb{E}[Z] = \sum_{m=1}^{\kappa} \Pr[Z \geq m] = \sum_{m=1}^{\kappa} (1 - P_c(m)) = \kappa - \sum_{m=1}^{\kappa} P_c(m). \quad (11)$$

One-shot list bound. Apply Lemma 1 to the size- κ list $\mathcal{L}_\kappa$. With $m = \kappa$ and $n - \kappa > \kappa$ (so $\nu = \log_2 \frac{n-\kappa}{\kappa} > 0$),

$$\log_2 n - I \leq 1 + P_c(\kappa) \log_2 \kappa + (1 - P_c(\kappa)) \log_2(n - \kappa).$$

Rearranging, and using $\log_2 n \geq \log_2(n - \kappa)$,

$$-(I + 1) \leq (\log_2(n - \kappa) - \log_2 n) + P_c(\kappa)(\log_2 \kappa - \log_2(n - \kappa)) \leq -P_c(\kappa) \nu,$$

so $P_c(\kappa) \leq \frac{I+1}{\nu}$. Because $\mathcal{L}_m \subseteq \mathcal{L}_\kappa$ for $m \leq \kappa$, $P_c(m) \leq P_c(\kappa) \leq \frac{I+1}{\nu}$, and plugging into (11),

$$\boxed{\mathbb{E}[Z] \geq \kappa \left[1 - \frac{I+1}{\nu} \right]_+}. \quad (12)$$

Here the positive part combines the derived inequality with $Z \geq 0$. This is the bound in Theorem 1.

(C) Corollaries. (i) *Endpoints.* The size-aware ledger reveals the size vector, which identifies the unique block of size L , so $I = \log_2 n$; then the agent archives $\{J^*\}$, freeing exactly L , giving $\tau = 1$, $Z = 0$, hence 0 recoveries and 0 recall errors. The content-only agent (no ledger) has $I = I_{\text{content}}$. Whenever $(I_{\text{content}} + 1)/\nu \rightarrow 0$, (12) gives $\mathbb{E}[Z]/\kappa \rightarrow 1$; fixed κ and σ with $n \rightarrow \infty$ is one such regime by Proposition 2. The pure size-blind case $I = 0$ is the special instance $\sigma \rightarrow \infty$.

(ii) *Threshold.* $\mathbb{E}[Z] \leq \delta\kappa$ forces, via (12), $\kappa(1 - \frac{I+1}{\nu}) \leq \delta\kappa$, i.e. $I \geq (1 - \delta)\nu - 1$.

(iii) *Price.* Before taking the positive part, the right-hand floor $\kappa(1 - \frac{I+1}{\nu})$ is affine in I with slope $-\kappa/\nu$. Thus the lower-bound floor changes at rate κ/ν per bit. In particular, requiring $\mathbb{E}[Z] \leq \kappa - \Delta$ forces $I \geq \Delta\nu/\kappa - 1$ bits.

From Z to recovery obligations and errors. Each archived load-bearing block is queried later. Any policy that answers all these queries correctly must make every such block available again, so the expected number of block-level recovery obligations is $\geq \mathbb{E}[Z]$. This counts blocks that must be restored, not file-read calls; a grouped payload read may satisfy several obligations. If at most r archived load-bearing blocks can be restored, at least $[\mathbb{E}[Z] - r]_+$ remain unavailable in expectation, each causing an exact-recall failure. All three size-aware costs are 0.

(D) Achievability (order-wise upper bound). Given a rate budget I , let $M := \min\{n, \lfloor 2^I \rfloor\}$ and consider a balanced M -bucket ledger: partition $[n]$ into M buckets, each of size at most $b := \lceil n/M \rceil$, and report the bucket Y containing J^* . Since Y is a deterministic function of J^* , $I(J^*; Y) = H(Y) \leq \log_2 M \leq I$, so this is a rate- $\leq I$ interface. The agent probes blocks inside the reported bucket in uniformly random order until the flag clears. Conditional on a bucket of size $b' \leq b$, the bulky block sits at a uniform position $P \in \{1, \dots, b'\}$. As in part (E), it frees L upon capture, so the number of load-bearing blocks archived is $\min(P - 1, \kappa) \leq P - 1$, giving

$$\mathbb{E}[Z | Y] \leq \mathbb{E}[P - 1 | Y] = \frac{b' - 1}{2} \leq \frac{\lceil n/M \rceil - 1}{2}.$$

The last bound holds for every bucket, and therefore also holds after averaging over Y . Because $\lceil x \rceil - 1 < x$, $\mathbb{E}[Z] < n/(2M)$. Therefore $I \geq \min\{\log_2 n, \log_2 \lceil n/(2\delta\kappa) \rceil\}$ suffices for $\mathbb{E}[Z] \leq \delta\kappa$. At $I = \log_2 n$, we have $M = n$, hence singleton buckets and $Z = 0$. For every fixed $\delta \in (0, 1)$, this sufficient rate and the necessary rate $(1 - \delta)\nu - 1$ are both $\Theta(\log_2(n/\kappa))$. If instead $\delta_n \rightarrow 0$ with $\log_2(1/\delta_n) = o(\nu)$, both rates are $(1 + o(1))\nu$.

(E) Why archive-in-an-order is without loss, and adaptivity is useless. Two reductions were used above. First, any admissible batch archive policy can be refined into a sequential order that examines the same selected blocks and stops as soon as overflow clears. This refinement has no larger load-bearing cost, so a lower bound proved for the more powerful sequential class also applies to batch policies. Second, adaptivity through the binary flag adds no usable information: before the bulky block is archived, every archived prefix of $t \leq \kappa - 1$ load-bearing blocks has freed $t\ell < L$, so the flag reads “over” deterministically and is independent of *which* blocks were chosen; after the bulky block is archived the freed size is $\geq L$ and the process stops. Thus the only J^* -information available to the ordering is Y , exactly as assumed, and Lemma 1 applies to the (Y, U) -measurable prefix lists $\mathcal{L}_m$. This is why the lower bound (12) holds for all adaptive content-only ($I = I_{\text{content}}$) and rate- I policies alike. $\square$

Remark 1 (Interpretation: dashboard bits versus block-level recovery). Theorem 1 identifies a sharp, order-wise phase transition in an information parameter: reducing the recovery burden below a fixed fraction of κ requires $\Theta(\log(n/\kappa))$ bits about which block to evict. The affine lower-bound floor changes at rate κ/ν per bit. The no-dashboard agent perceives a genuine, noisy size signal from content, but Proposition 2 places that signal at $I_{\text{content}} \leq (\ln 2)(\log_2 \kappa)^2/\sigma^2$ bits. For fixed κ and σ this is constant in n , so it falls below the $\log_2(n/\kappa)$ make-room threshold once the workspace is large, while the full ledger reaches the $I = \log_2 n$ end. The theorem itself is distribution-free in Y ; Proposition 2 supplies an analyzable content-only endpoint. The empirical bridge is the corresponding operational quantity—the model’s size-*ranking* ability: the pairwise size-comparison accuracy of Appendix I measures how well content localizes the block to evict, and the dashboard drives it toward the $I = \log_2 n$ end (median relative total-size error 0.43–0.84 without the ledger, collapsing to 0 with it; Table 5). The 255/57 vs. 69/105 archive/retrieve split (Figure 8b) is the over-archive-under-retrieve signature the theorem predicts.

Method	Training-free	Model-agnostic	Agent-controlled	Exact recovery	Context dashboard
ReAct (append until truncation)	✓	✓	✗	✗	✗
Tool-result clearing	✓	✓	✗	✗	✗
Stale-observation masking	✓	✓	✗	✗	✗
Active Context Compression	✓	✓	✓	✗	✗
Skeleton compression	✓	✓	✗	✗	✗
Claude Code	✓	~	✓	~	✗
Learned compression (CAT, RL budget)	✗	✗	✓	✗	✗
Context-Folding	✗	✗	✓	✗	✗
LongSeeker	✗	✗	✓	✗	✗
GenericAgent	✗	✗	✓	✗	✗
VISTA (ours)	✓	✓	✓	✓	✓

Table 6: Method capability comparison. Capability summary against the design properties of Section 2; VISTA uniquely combines agent-facing state with exact recovery.

E Method Capability Comparison

Table 6 summarizes the evaluated baselines and the learned-compression family against the design properties of Section 2. It isolates the two properties that distinguish VISTA: an agent-facing context dashboard and byte-exact recovery of externalized evidence. The marks are a coarse capability summary, not a performance claim, and partial marks reflect mechanisms that hold the property only in part, such as Claude Code, which keeps files on disk but still summarizes the active context.

F Implementation Details

This section reports the exact run configuration, the verbatim prompt the agent receives, the dashboard format, and the context tool definitions, so the setting can be reproduced without access to our harness.

F.1 End-to-End Context Loop

Algorithm 1 expands the compact loop in Section 2. The recoverable store A_t includes agent-archived payloads and any raw tool result externalized by the final wire-payload guard; explicit deletion removes a payload from this store.

Algorithm 1 Self-managed context loop

Require: task block b_{task} , budget B , environment tools $\mathcal{T}_{\text{env}}$, context tools $\mathcal{T}_{\text{ctx}}$
Ensure: final answer and workspace trajectory $W_{1:T}$

- 1: $V_0 \leftarrow \{b_{\text{task}}\}; A_0 \leftarrow \emptyset$
- 2: **while** the task is unfinished **do**
- 3: Apply the budget guard: admit fitting results; offload eligible visible raw results or reject an unadmittable new result
- 4: $D_t \leftarrow \text{Dashboard}(V_t, A_t, B); \tilde{C}_t \leftarrow \text{Assemble}(V_t, A_t) \cup \{D_t\}$
- 5: $C_t \leftarrow \text{preflight}(\tilde{C}_t, B)$
- 6: Use $\mathcal{T}_{\text{ctx}}$ only if $|\tilde{C}_t| > B$; otherwise use $\mathcal{T}_{\text{env}} \cup \mathcal{T}_{\text{ctx}} \cup \{\text{answer}\}$
- 7: $a_t \leftarrow \text{LLM}(C_t)$
- 8: If $a_t = \text{read_path}(h, q)$, read the requested exact bytes and admit the returned content subject to the same budget guard
- 9: Else if $a_t = \text{archive}(\mathcal{S}, \rho)$, replace $\mathcal{S}$ by handles and store exact payloads in A_t
- 10: Else if $a_t = \text{delete}(\mathcal{S})$, remove $\mathcal{S}$ and its stored payloads
- 11: Else if $a_t \in \mathcal{T}_{\text{env}}$, execute it; if $a_t = \text{answer}$, **return** the answer
- 12: $W_{t+1} \leftarrow (V_t, A_t)$
- 13: **end while**

Benchmark setting. LOCA-Bench evaluates online tool agents under controllable context growth: the agent must continue acting while earlier reasoning, tool calls, and observations remain in or are

externalized from the working context. We evaluate 75 task configurations; unless otherwise stated, accuracy is solved tasks over all 75, with errors and timeouts counted as failures. AMA-Bench is used as a secondary generalization benchmark. Its episodes provide a completed trajectory and ask questions about past events, causal relations, and state changes. This is not the native online-control setting for VISTA; it tests whether the same context-management layer can be adapted into trajectory memory. We use the benchmark’s two-stage memory interface. During memory construction, the completed trajectory is replayed step by step as a growing conversation: each action and observation becomes a workspace block, the future questions are hidden, and the replayed agent may archive exact payloads when the workspace budget becomes tight. During retrieval, VISTA assembles the resulting workspace, dashboard, construction events, and recoverable archive handles for the current question. Thus the AMA-Bench result evaluates a replayed VISTA workspace as offline memory [35], instead of simply placing the full trajectory in the model prompt.

Run configuration. VISTA is integrated into the LOCA-Bench harness and invoked as a strategy (`loca run -s self_managed`) with no training and no per-model tuning. The main results use `gemini-3-flash` at a 128K budget (`max-context-size = 128,000`); the cross-backbone runs reuse the same strategy unchanged on `claude-sonnet-4-5`, `deepseek-v4-pro` (open-weight), and `glm-5` (open-weight). Two flags define the full method: `SM_STRICT_LONG_CONTEXT=1` enforces a hard budget instead of issuing a soft warning, and `SM_BETTER_DASHBOARD=1` selects the factual ledger dashboard below. The ablations toggle single flags from this base, for example `SM_DISABLE_ARCHIVE`, `SM_DISABLE_AGENT_ARCHIVE` (fixed archive policy), and `SM_ENABLE_STATE_BOARD` (status-board variant). Per-task timeout is 1800 seconds and reasoning effort is medium across all backbones.

Baseline definitions. We organize the LOCA-Bench baselines by who makes the keep-or-drop decision. Fixed external policies include ReAct, which appends until truncation; Tool-result Clearing, which removes old tool-result/tool-call pairs after the prompt crosses a threshold; and fixed stale masking, which masks old tool observations while preserving the assistant reasoning and tool-call skeleton. Agent-mediated baselines still reduce context irreversibly. SLIM [42], reproduced from its public release, periodically summarizes older context once the budget is exceeded. Active Context Compression [26] asks the agent to write and prune its own knowledge blocks. A structured-compression baseline preserves a compact skeleton of prior context, following the design of context-as-a-tool compressors [16]. Learned members of this family generally do not release trained checkpoints and, in many cases, do not release code, so we reproduce the training-free methods directly and follow the published inference-time design for the rest without introducing a trained policy. Claude Code is the Claude Code command-line agent at the CLI release of May 6, 2026, included as a strong practical agent with mature tool-use and context-handling heuristics. We retain its complete released CLI harness and replace only the MCP task tools with the benchmark-native equivalents. These baselines cover deletion, masking, summarization, self-compression, and structured compression. None combines agent-facing context-state metadata with exact evidence recovery. On AMA-Bench, the EMem-style and Mem0-style rows are local adapters implemented for this harness and should be read as engineering baselines, not official reproductions.

Baseline reproduction details. SLIM and Active Context Compression are faithful reproductions of training-free published methods, run with the procedure described by their authors and triggered at the same 128K budget used for every method. SLIM periodically summarizes older context once the budget is exceeded, and Active Context Compression runs the explore, write a knowledge block, then prune the raw history loop. The structured-compression baseline is inspired by context-as-a-tool compressors but is not a faithful reimplement, since that method is learned and we run no trained policy. For harness-native baselines we vary only the context-management mechanism and hold the agent loop, tools, budget, backbone, and scoring fixed. For Claude Code, the released CLI harness is held intact apart from substituting the benchmark-native task tools for the MCP task tools; task instances and scoring remain unchanged.

Context-management protocol. The agent receives the following instruction block appended to the task prompt, together with a budget notice. It is identical across backbones.

Context-management protocol (verbatim)

CONTEXT MANAGEMENT PROTOCOL:

A <context_workspace_status> dashboard is shown every turn as a compact map of context blocks. Use context tools only when clearly needed. Do not archive, delete, or offload content solely because it is old, large, or listed in context metadata; leave content visible when the context budget is sufficient. Large payloads may be represented by placeholders; inspect originals only when needed. Use ordinary file/terminal/python tools, source metadata, and any in-context payload placeholders to inspect external evidence when details are needed. For structured data or calculations, use the source file, source tool, or query directly. Do not copy table, CSV, or JSON rows from the conversation into code.

You need to complete the task within the following context window size:
<budget:token_budget>128000</budget:token_budget>

Dashboard format. Each turn the harness injects a <context_workspace_status> block. It is a budget bar followed by one ledger row per block, with columns ID, approximate tokens, age (root-turn distance, where 0r is newest), type, compression level, parent, and status (visible, pinned, archived, or offloaded_placeholder). The instance below shows the same compact column subset used in Figures 1 and 3; the full renderer additionally prints the compression Level and Parent columns described above.

Dashboard instance shown to the agent

```
## Context Budget
[#####-----] 62% (~79,400 / 128,000 tokens)
overhead ~6,200 | conversation ~71,900 | dashboard ~1,300

## Context Blocks (Age = root-turn distance; 0r newest)
ID      ~Tok  Age  Type              Status
-----
B1       120   8r  user_message      pinned
B2    18,400   7r  tool_call          archived
B3     2,150   6r  assistant_message  visible
B5    14,800   3r  tool_call          visible
B6     9,300   2r  tool_call          visible
B9     1,070   0r  assistant_message  visible
```

Context tool definitions. The agent acts on the workspace with two tools. Archiving replaces a block with a compact handle and returns the payload file path; the agent recovers byte-exact content by reading that path with ordinary file or terminal tools, so recovery is a normal read, with no dedicated decompressor.

Context tool definitions (docstrings)

```
context_workspace_archive(block_id: str, replacement: str = "") -> str
    Replace one or more blocks with compact indexes. The original content is
    stored externally as a payload file. Operations are block-level: only listed
    block IDs are archived. Returns the payload file path for later recovery.
    block_id: Block IDs, ranges, or group IDs, e.g. "B3", "B3,B4",
              "B10-B20", or "G2".
    replacement: Short index text for the archived block(s).

context_workspace_delete(block_id: str, reason: str) -> str
    Permanently remove one or more blocks. Deleted content cannot be recovered.
    block_id: Block IDs, ranges, or group IDs.
    reason: Short reason why the content has no future task value.
```

Large tool results are stored as external transcript payloads with compact placeholders. These payloads record what a tool returned to the model, not a complete source database; if a transcript is truncated or paginated, the agent must query the original source tool for complete data.

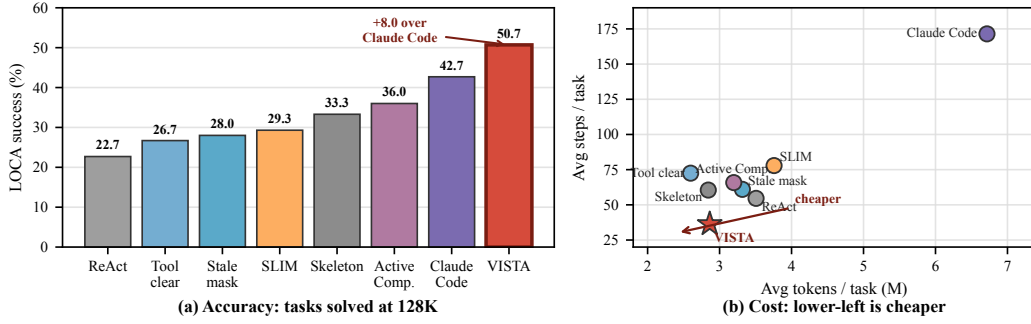


Figure 11: Expanded LOCA result at 128K. Tasks solved, tokens, and steps for the main LOCA-Bench comparison.

Family	Method	Correct	Acc.	Timeout	Error	Steps	Tokens	Mgmt. events	Notes
No CM	ReAct	17	22.7	0	0	54.6	3.51M	636 trims	full 75
Deletion	Tool-result Clearing	20	26.7	0	0	72.9	2.60M	1,968 clears	full 75
Masking	Fixed stale masking	21	28.0	–	–	61.0	3.32M	–	full 75
Summary	SLIM	22	29.3	–	–	77.9	3.76M	–	full 75
Self-compression	Active Context Compression	27	36.0	–	–	65.8	3.20M	–	full 75
Structured compression	Skeleton compression	25	33.3	–	–	60.5	2.84M	–	full 75
Agent CLI	Claude Code	32	42.7	0	22	171.5	6.72M	–	full 75
Ours	VISTA	38	50.7	20	0	36.4	2.86M	69 archive / 105 read	full 75

Table 7: Dense LOCA run ledger. 128K main comparison with execution cost and method-specific context-management events.

Comparison	Both	Base.	VISTA	Neither
Fixed stale masking	18	3	20	34
SLIM	17	5	21	32
Active Context Compression	20	7	18	30
Claude Code	25	7	13	30

Table 8: Outcome transitions. Pairwise 128K LOCA-Bench split against each baseline: both solve, baseline-only, VISTA-only, and neither.

G Evaluation Details

For LOCA-Bench, we use the independently released public task suite and evaluation protocol without modification. It contains 75 online tool-task configurations with controllable context growth, where prior reasoning, tool calls, and observations accumulate until context management becomes central. We report task success, count errors and timeouts as incorrect, and compute average steps and tokens over task rows present in each run log. These cost values therefore describe observed execution cost, not cost conditioned on success. We additionally log archive and recover/read events for VISTA variants.

Figure 11 and Table 7 expand the 128K LOCA-Bench comparison, Table 8 gives the pairwise rescued-task split, and Table 9 gives exact counts for the context-growth sweep in Figure 4.

For BrowseComp-Plus, we evaluate deep-research retrieval with DeepSeek-V4-Pro on an $N=150$ subset and report judged Pass@1 with one sampled answer. The agent searches the benchmark’s fixed corpus [3], so evidence is scattered across retrieved passages and the transcript grows through repeated retrieval. To expose context management, we use a deliberately tight active window ($W=12K$ tokens per call) and total budget ($B=160K$), chosen so early evidence can be evicted before synthesis.

For GAIA, we use a fixed random 165-question subset of the public validation split. We preserve the original question text and attached files, require the official FINAL ANSWER: format, and score with quasi-exact match. All methods use DeepSeek-V4-Pro, real web/search/file tools, $W=12K$, and $B=80K$.

Method	Setting	Success	Avg. steps	Avg. tok.	Notes
VISTA	8K	86.7	16.7	0.44M	complete
VISTA	16K	84.0	16.2	0.52M	complete
VISTA	32K	70.7	17.4	0.87M	complete
VISTA	64K	61.3	21.2	1.38M	complete
VISTA	96K	57.3	29.7	2.32M	complete
VISTA	128K	50.7	36.4	2.86M	complete
VISTA	256K	32.0	43.1	3.51M	complete
ReAct	8K	84.0	27.9	0.68M	complete
ReAct	16K	74.7	24.9	0.80M	complete
ReAct	32K	65.3	25.3	1.02M	complete
ReAct	64K	52.0	29.6	1.70M	complete
ReAct	96K	36.0	39.5	2.79M	complete
ReAct	128K	22.7	54.6	3.51M	complete
ReAct	256K	12.0	82.6	5.80M	complete

Table 9: Observed pressure sweep. Matched VISTA and ReAct runs over the full 75-task suite.

Method	Embodied		Game		OpenQA		Software		Text2SQL		Web	
	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
AMA	0.678	0.489	0.814	0.407	0.817	0.249	0.565	0.150	0.838	0.395	0.782	0.272
VISTA	0.683	0.525	0.747	0.443	0.731	0.386	0.567	0.238	0.846	0.384	0.763	0.341

Table 10: AMA domain breakdown. By-domain accuracy and F1 for AMA and the VISTA trajectory-memory adaptation.

Case	Baselines failed		VISTA steps	Archive/read evidence	evi-	Evaluation signal
NHL B2B schedule analysis	ReAct, SLIM	Tool-clear,	64	5 archive calls; 17 payload reads/uses		CSV and Google Sheet correct; HA/AH/HH/AA counts verified
WooCommerce low-selling products	ReAct, Claude-style, Tool-clear, SLIM		38	4 archive calls; 3 payload reads/uses		Correct products moved; subscriber emails sent
Canvas final-exam schedule	ReAct, Claude-style, Tool-clear, SLIM		17	1 archive call for course announcements		Final Excel schedule accepted
NLP course reminders	ReAct, Claude-style, Tool-clear, SLIM		31	1 archive call for large roster table		Correct students emailed; dropped/submitted students excluded

Table 11: Rescued-task case studies. Tasks VISTA solves where multiple baselines fail, with archive and payload-use counts.

For AMA-Bench, each of the 208 episodes contains 12 open-ended questions, giving 2496 judged QA pairs. We report judge accuracy and token-level F1 following the benchmark harness. Runtime per episode is measured for generation; judge time is reported separately in the analysis files.

H Limitations

VISTA supplies the missing proprioceptive signals, but it does not guarantee the agent uses them well. A model can still misread the dashboard, archive evidence it later needs, or recover a payload too late. The elicitation view also predicts a floor: a model with little latent context-management skill has little for the interface to unlock, and GLM-5 and DeepSeek-V4-Pro show the smallest gains in our four-backbone comparison. We test four backbones; mapping the low-capability end of this curve remains open. We also do not test transferable adversarial inputs [32], poisoned external evidence [33], or generative trigger settings [39].

VISTA complements post-training; it is not an alternative to it. Training improves what an agent does with context-state information, while the dashboard supplies exact runtime state that is not explicitly available in the prompt. Section 2.5 demonstrates this compatibility in a Qwen3-8B transfer setting.

Scaling post-training across the main multi-benchmark and multi-backbone suite, enriching the ledger with predicted relevance, and comparing against learned compression managers or memory-action policies [43, 44, 48] are natural next steps. Finally, the EMem-style and Mem0-style AMA-Bench rows are local adapters, not official implementations, so they support diagnosis but are not final claims against those systems; AMA-Bench remains a transfer test, not a primary benchmark.

Our main conclusions rest on large task-level gaps over fixed evaluation sets. Most benchmark cells use one sampled trajectory per task; repeated-seed evaluation remains future work for run-to-run variance and small differences.

I Proprioceptive-Blindness Diagnostic

This appendix documents the diagnostic behind Table 5. The goal is to measure directly whether a backbone can read its own context state, separating perception from skill.

Data. We anchor on the first archive event of each real LOCA-Bench run, the moment the agent itself decided to externalize content. We take the accumulated transcript just before that call as the snapshot, treating each message as one block. The runtime dashboard is not persisted in the transcript, so the stored messages are already free of the live ledger; we additionally strip the three injected artifacts that would leak state, namely the context-management protocol header, the hard-limit rejection notices that print token counts, and archived-block placeholders. A scan over all anchored snapshots confirms no residual token, budget, or usage strings remain, and there is no per-block usage annotation. Twenty-nine runs contain an archive; we cap each snapshot at 100K tokens by dropping trailing blocks so it fits every backbone window, and compute ground truth with the same tokenizer used by the harness.

Questions and conditions. We ask three quantities, each in its own request so the measurements stay independent. *Total size*: estimate the token count of the whole transcript. *Block size*: estimate the token count of four sampled blocks. *Pairwise*: for sampled block pairs, say which is larger, reported on the hard subset within $2\times$ in true size. The *-dash* condition shows the cleaned transcript only; the *+dash* condition prepends the factual ledger (the columns of the implementation-details appendix). Size answers are scored as median relative error and pairwise as accuracy against the larger block. We run Gemini-3-Flash, Claude-Sonnet-4.5, DeepSeek-V4-Pro, and GLM-5 with greedy decoding. Claude-Sonnet-4.5 returns valid structured output slightly less often than the other three, but the qualitative gap and its closure with the dashboard hold for every backbone. These probes establish context-state blindness in the token-magnitude dimension that directly governs budgeted context management; the interface exposes recency and archive status alongside this measured signal.

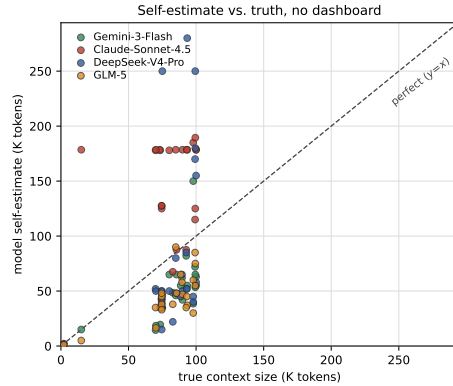


Figure 12: Proprioceptive-blindness diagnostic. Without the dashboard, self-estimated context size is poorly calibrated; the factual ledger closes the gap.